



AKTU

B.Tech 6th Semester
CS IT & Allied



Computer Networks

Unit-4: Lecture 1

Today's Target

- Transport Layer ✓
- TCP Services and Features ✓
- TCP Header Format ✓
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified

Computer Networks

Unit-4 : Syllabus

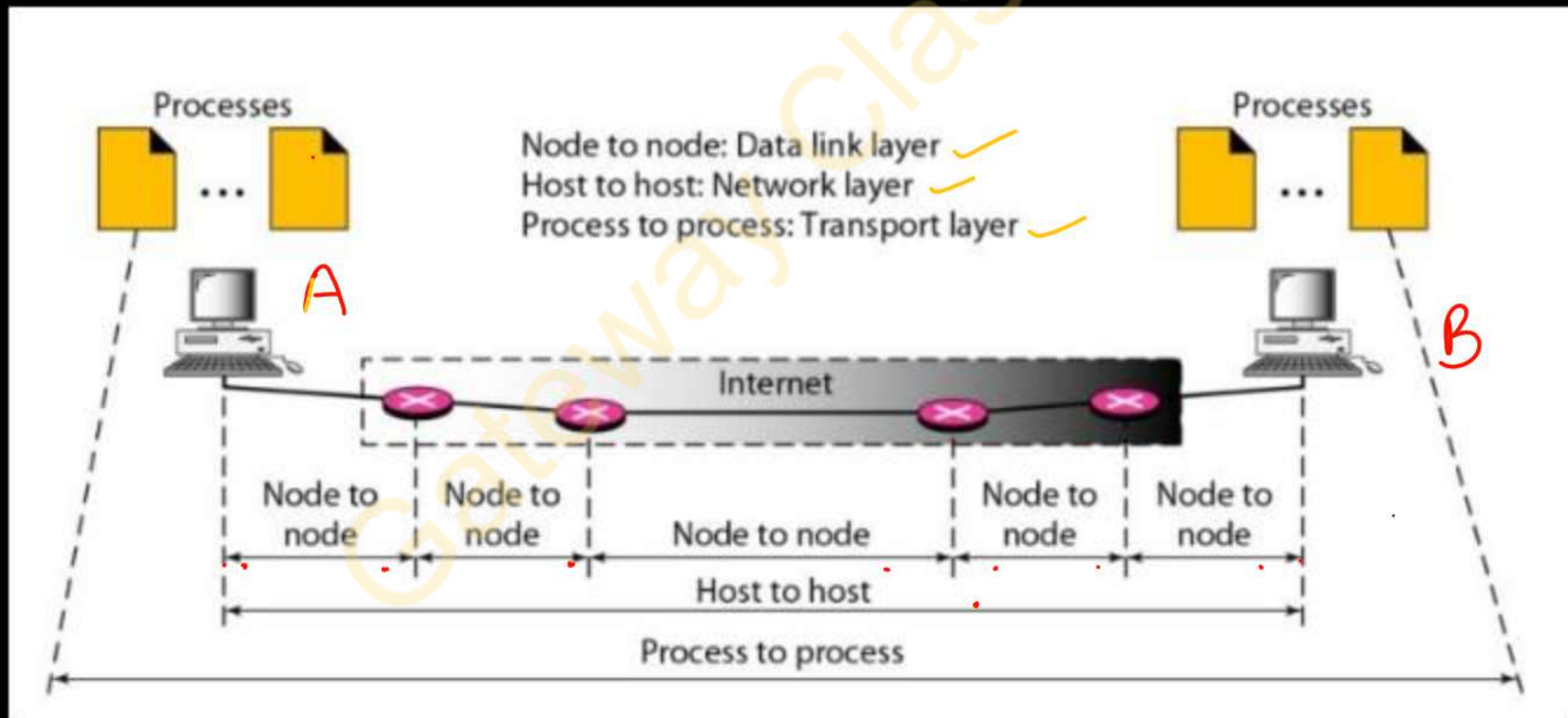
Transport Layer: Process-to-process delivery, Transport layer protocols (UDP and TCP), Multiplexing, Connection management, Flow control and retransmission, Window management, TCP Congestion control, Quality of service.

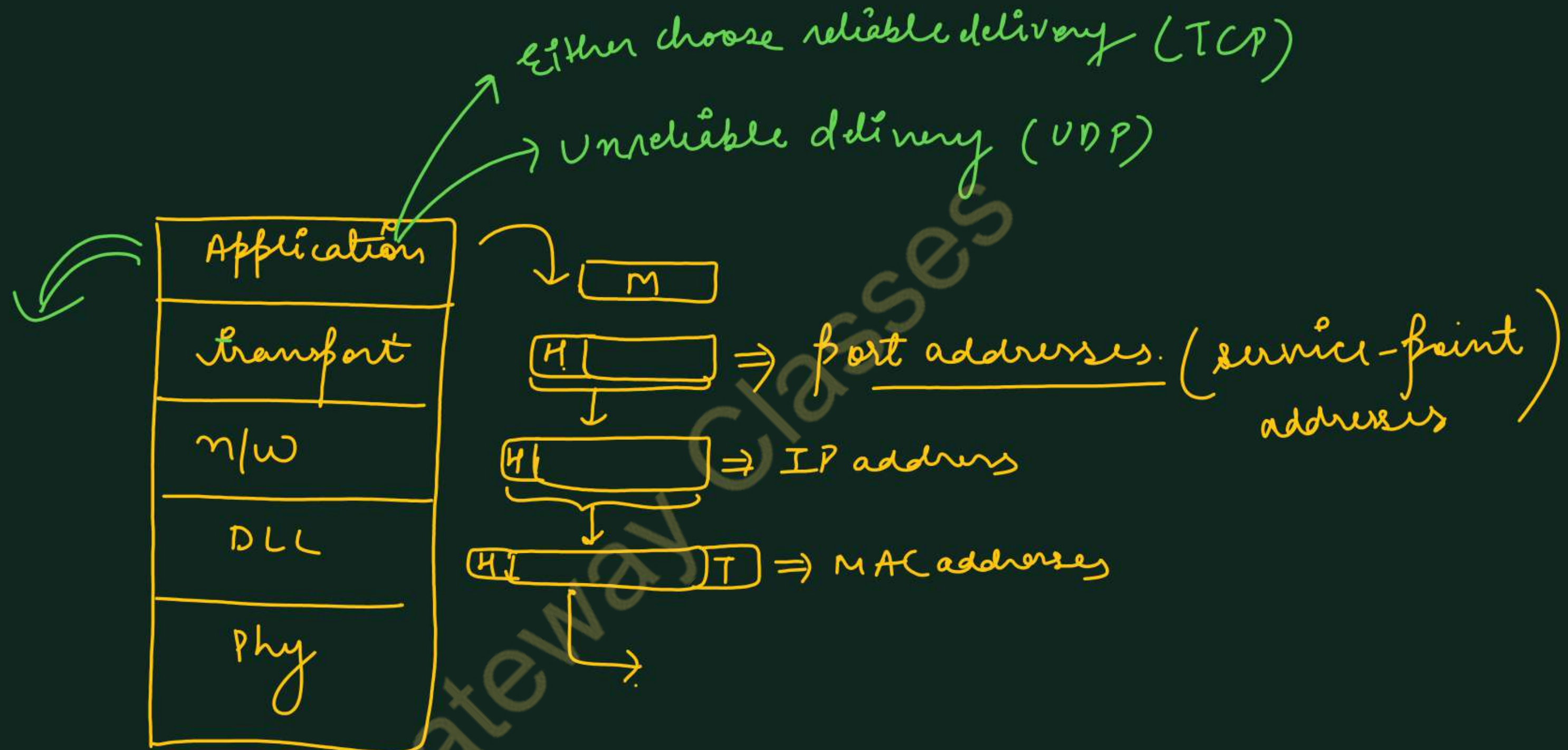
Gateway Classes

Transport Layer

- Transport layer is responsible for process-to-process delivery of the entire message.
- Ensures that the whole message arrives intact and in order, along with **error control and flow control** at the source-to-destination level.

The transport layer is responsible for the delivery of a message from one process to another.





- Computers often run several programs at the same time. So, source to destination delivery means delivery from a specific process on one computer to a specific process on the other.
- Transport layer header include a service-point address(Port address) of source and destination.
- A transport layer protocol can be either connectionless or connection-oriented. (TCP) Reliable
(UDP)
- A connectionless transport layer treats each segment as an independent packet.
- A connection-oriented transport layer makes a connection with the transport layer at the destination machine first before delivering the packets. After all data is transferred, connection is terminated.
- Like the data link layer, the transport layer may be responsible for flow and error control. However, flow and error control at this layer is performed end to end rather than across a single link.

Port numbers:

In the Internet model, the port numbers are 16-bit integers between 0 and 65,535.

The client program defines itself with a port number, chosen randomly by the transport layer software running on the client host.

The server process port number cannot be chosen randomly.

The Internet has decided to use universal port numbers for servers; called well-known port numbers.



1. Well-known ports: The ports ranging from 0 to 1023 are assigned and controlled by IANA. These are the well-known ports.
2. Registered ports: The ports ranging from 1024 to 49,151 are not assigned or controlled by IANA. They can only be registered with IANA to prevent duplication.
3. Dynamic ports: The ports ranging from 49,152 to 65,535 are neither controlled nor registered. They can be used by any process. These are the ephemeral ports.

Connectionless Versus Connection-Oriented Service

A transport layer protocol can either be connectionless or connection-oriented.

Connectionless Service:

- Packets are sent from one party to another without connection establishment or connection release.
- The packets are not numbered; they may be delayed or lost or may arrive out of sequence.
- There is no acknowledgment either. User datagram protocol(UDP), is connectionless.

Connection~Oriented Service:

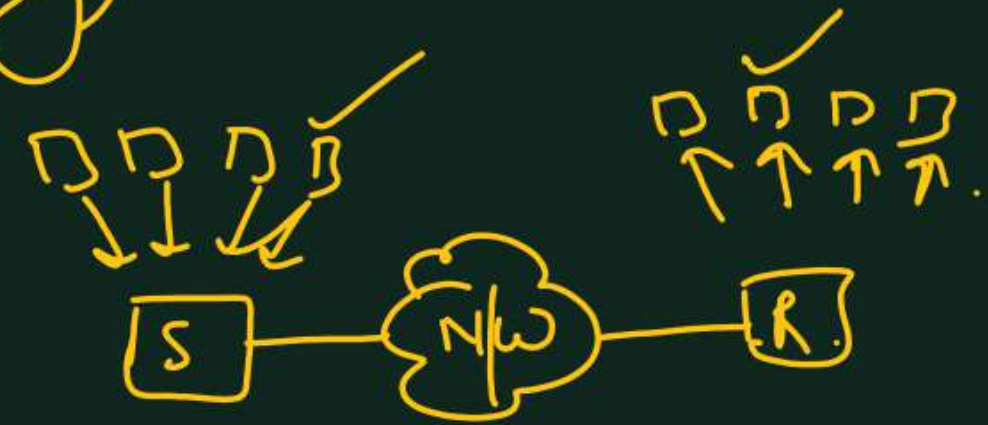
- A connection is first established between the sender and the receiver.
- Data are transferred.
- At the end, the connection is released. TCP and SCTP are connection-oriented protocols.

Reliable Versus Unreliable:

- The transport layer service can be reliable or unreliable.
- If the application layer program needs reliability, we use a reliable transport layer protocol by implementing flow and error control at the transport layer.
- If application program does not need reliability because it uses its own flow and error control mechanism or it needs fast service or the nature of the service does not demand flow and error control (real-time applications), then an unreliable protocol can be used.
- UDP is connectionless and unreliable.
- TCP and SCTP are connection oriented and reliable.

Responsibilities of Transport Layer

- 1) End-to-End delivery
- 2) Connection oriented and connectionless service to Applications
- 3) Service point Addressing (Port Addressing)
- 4) Segmentation and Reassembly.
- 5) Multiplexing and Demultiplexing →
- 6) error control
- 7) flow control
- 8) congestion control



Transmission Control Protocol (TCP)

- TCP is a process-to-process protocol.
- TCP is a connection oriented protocol; it creates a virtual connection between two TCPs to send data.
- TCP uses flow and error control mechanisms at the transport level.
- It adds connection-oriented and reliability features to the services of IP.

TCP is called a connection-oriented, reliable transport protocol.

TCP Services:

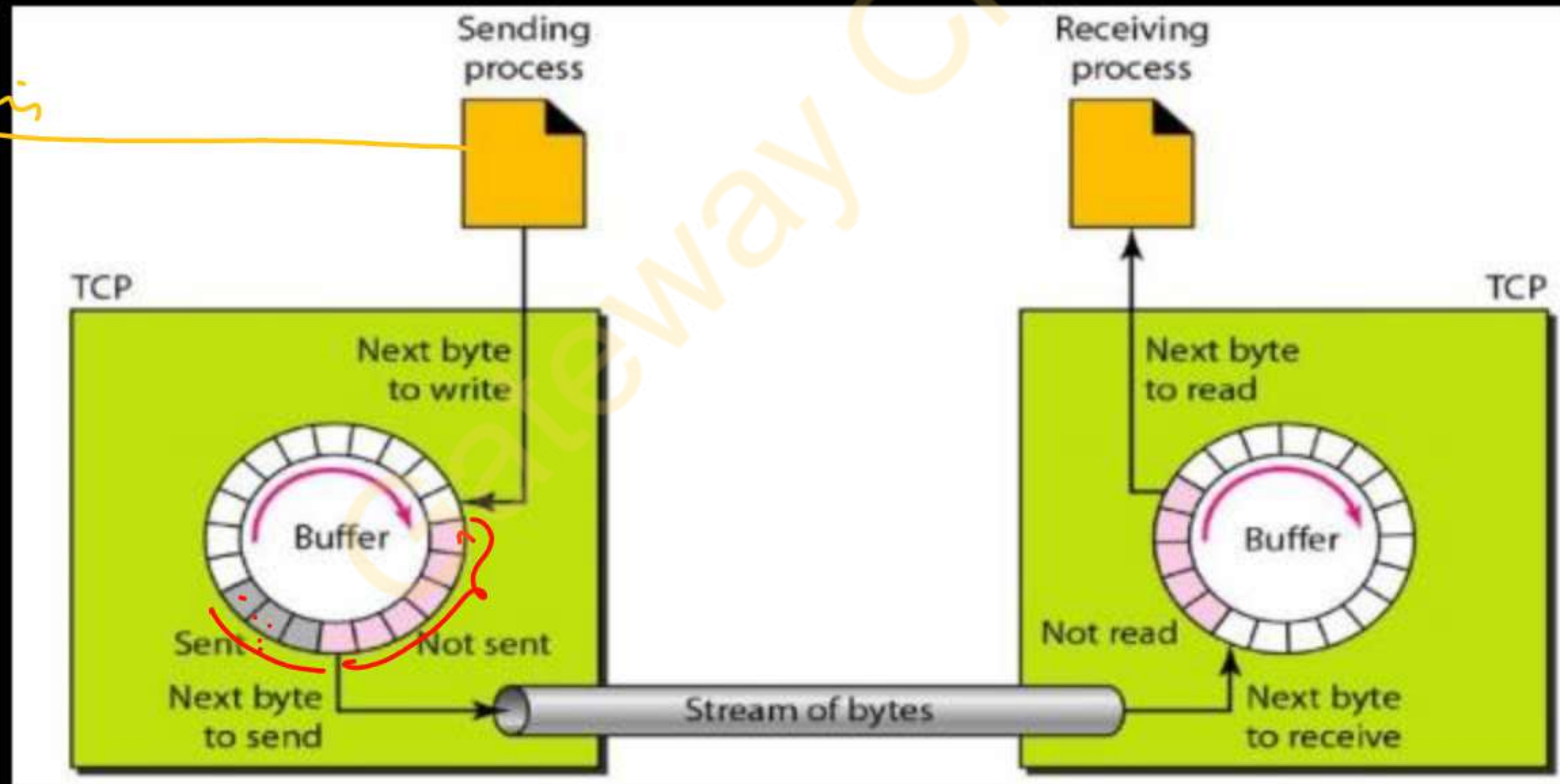
1. **Process-to-Process Communication**
2. **Stream Delivery Service:**
 - TCP allows sending process to deliver data as a stream of bytes and allows the receiving process to obtain data as a stream of bytes.

3. Sending and Receiving Buffers:

- As sending and the receiving processes may not write or read data at the same speed, TCP needs buffers(sending buffer and the receiving buffer) for storage.
- One way to implement a buffer is to use a circular array of 1-byte.
- Normally buffers are hundreds or thousands of bytes, and may be of different sizes.
- TCP keeps sent bytes in the buffer until it receives an acknowledgment.

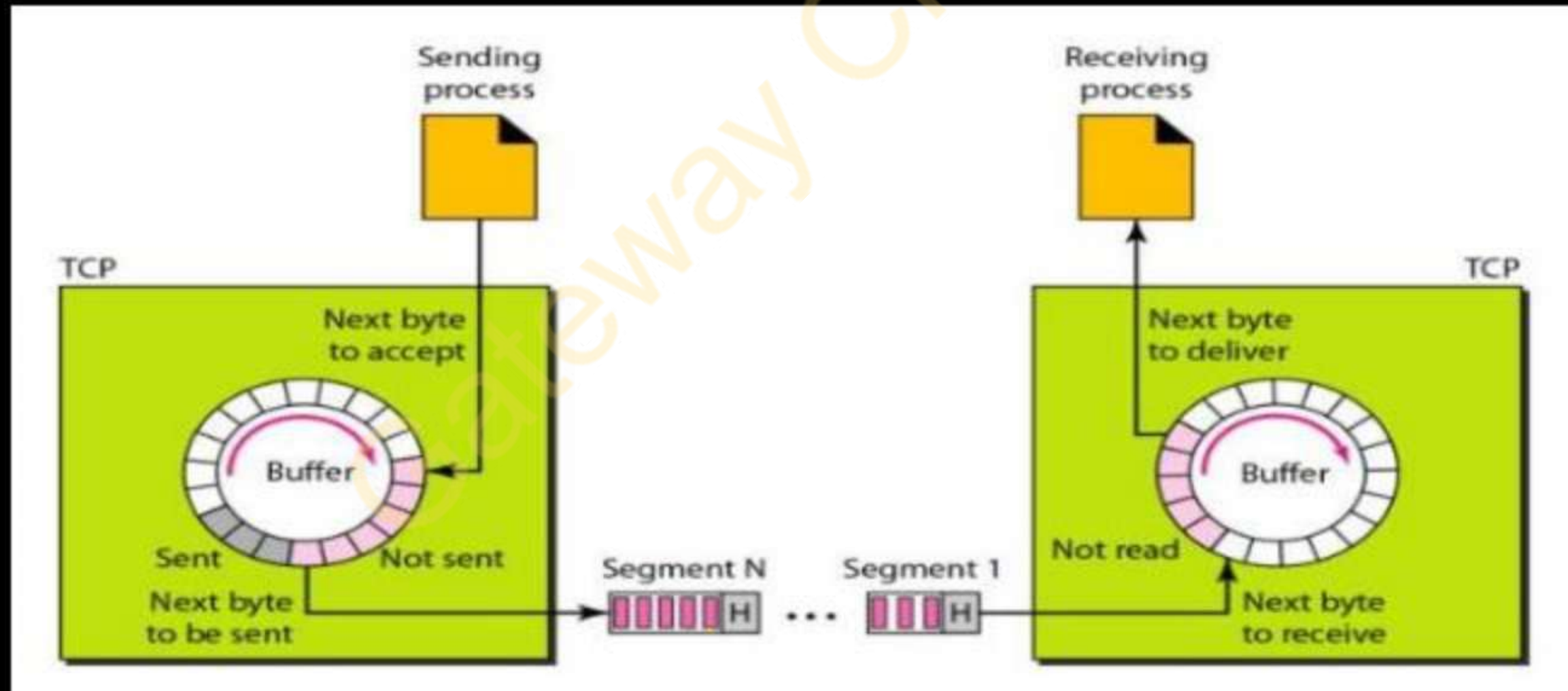
Application
layer.

Transport
layer.



4. Segments:

- IP layer, as a service provider for TCP, needs to send data in packets, not as a stream of bytes.
- TCP groups a number of bytes into a segment. Segments are not necessarily same size.
- TCP adds a header to each segment (for control purposes) and delivers it to the IP layer where they are encapsulated in IP datagrams and transmitted.
- Segments may be received out of order, lost, or corrupted and resent. All these are handled by TCP.



5. Full-Duplex Communication:

- TCP offers full-duplex service, in which data can flow in both directions at the same time.

6. Connection-Oriented Service:

- When a process at site A wants to send/receive data from another process at site B.
 1. The two TCPs establish a connection between them.
 2. Data are exchanged in both directions.
 3. The connection is terminated.
- This is a virtual connection, not a physical connection.
- TCP accepts the responsibility of delivering the bytes in order to the other site.

7. Reliable Service:

- It uses an acknowledgment mechanism to check the safe arrival of data.

TCP Features

Numbering System:

- There are two fields called the sequence number and the acknowledgment number.
- These two fields refer to the byte number and not the segment number.
- TCP numbers all data bytes that are transmitted in a connection. Numbering is independent in each direction.
- When TCP receives bytes of data from a process, it stores them in the sending buffer and numbers them. The numbering does not necessarily start from 0. Instead, TCP generates a random number between 0 and $2^{32} - 1$ for the number of the first byte.
- Example: if the random number happens to be 1057 and the total data to be sent are 6000 bytes, the bytes are numbered from 1057 to 7056.
- After bytes have been numbered, TCP assigns a sequence no. to each segment that is being sent. The sequence number for each segment is the number of the first byte carried in that segment.

1057 | 1062
[0 0 0 0 0 0] → seq no = 1062

Example:

Suppose a TCP connection is transferring a file of 5000 bytes. The first byte is numbered 10,001.

What are sequence numbers for each segment if data are sent in five segments, each carrying 1000 bytes?

Solution

The following shows the sequence number for each segment:

- ✓ Segment 1 Sequence Number: 10,001 (range: 10,001 to 11,000)
- ✓ Segment 2 Sequence Number: 11,001 (range: 11,001 to 12,000)
- ✓ Segment 3 Sequence Number: 12,001 (range: 12,001 to 13,000)
- ✓ Segment 4 Sequence Number: 13,001 (range: 13,001 to 14,000)
- ✓ Segment 5 Sequence Number: 14,001 (range: 14,001 to 15,000)

- Communication in TCP is full duplex; after making connection, both parties can send/receive data at the same time.
- Each party numbers the bytes, usually with a different starting byte number.
- The sequence number in each direction shows the number of the first byte carried by the segment.
- Each party also uses an acknowledgment number to confirm the bytes it has received.
- The value of ack field in a segment defines the number of next byte a party expects to receive.
- The acknowledgment number is cumulative.

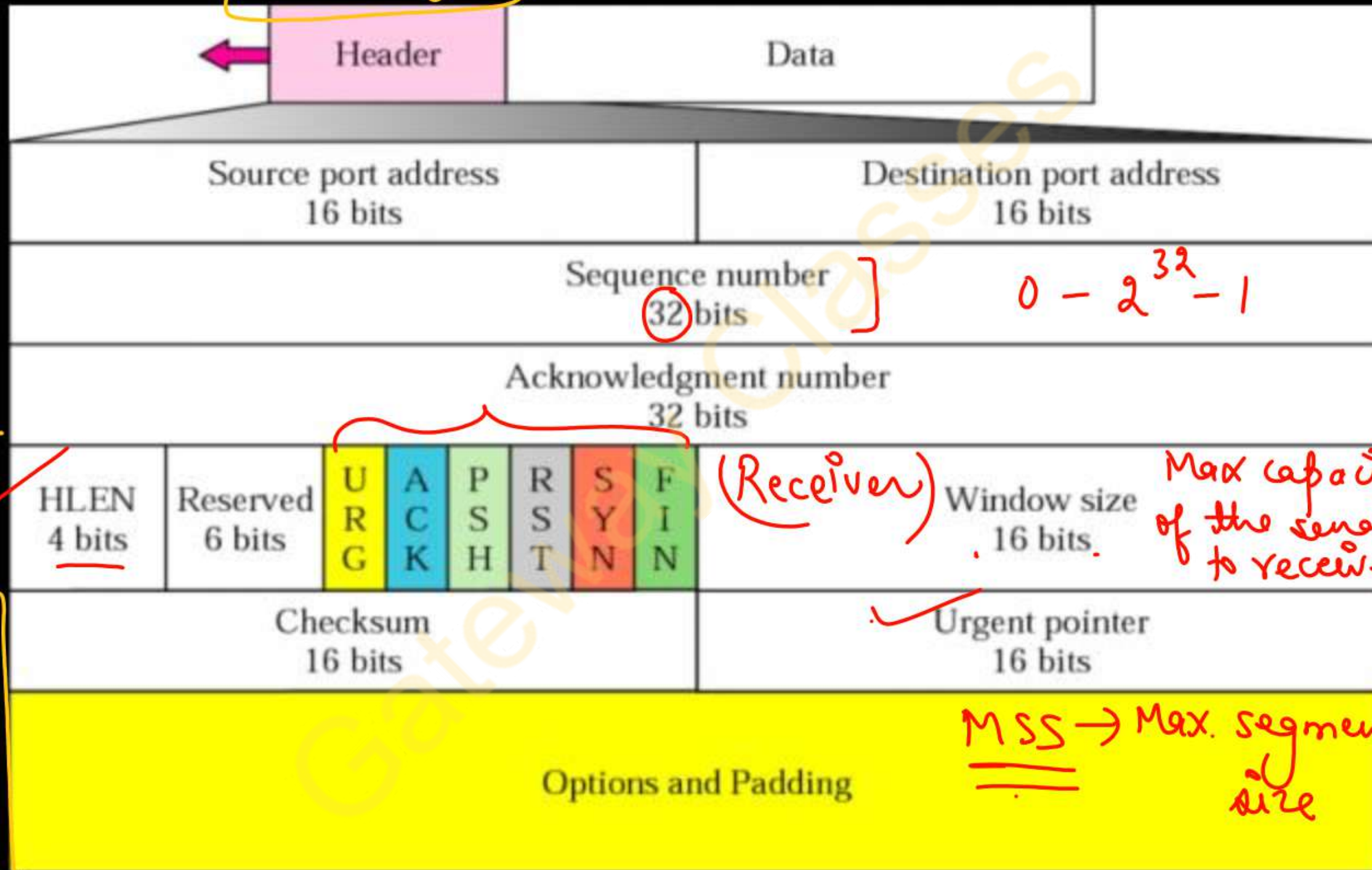
Flow Control: The numbering system allows TCP to use a byte-oriented flow control.

Error Control: Although error control considers a segment as the unit of data for error detection (loss or corrupted segments), error control is byte-oriented.

Congestion Control: TCP takes into account congestion in the network. The amount of data sent by a sender is not only controlled by the receiver (flow control), but is also determined by the level of congestion in the network.

TCP segment format

20-60 bytes



00000000

0000 → 0 X
 0001 → 1 X
 0010 → 2 X
 0011 → 3 X
 0100 → 4 X

1111 15

MSS → Max. segment size

The segment consists of a 20- to 60-byte header, followed by data from the application program.

The header is 20 bytes if there are no options and up to 60 bytes if it contains options.

➤ Source port address:

- 16-bit field defines port number of application program in the sending host.

➤ Destination port address:

- 16-bit field defines the port number of application program in receiving host.

➤ Sequence number:

- 32-bit field defines number assigned to first byte of data contained in this segment.
- During connection establishment, each party uses a random number generator to create an initial sequence number (ISN), which is usually different in each direction.

➤ Acknowledgment number:

- 32-bit field defines the byte number that receiver of the segment is expecting to receive from the other party.
- If the receiver of the segment has successfully received byte number x from the other party, it defines $x + 1$ as the acknowledgment number.
- Acknowledgment and data can be piggybacked together.

➤ Header length:

- This 4-bit field indicates length of TCP header in 4-byte words .

$$\text{Min} \Rightarrow 5 \times 4 = 20$$

$$\text{Max} \Rightarrow 15 \times 4 = 60$$

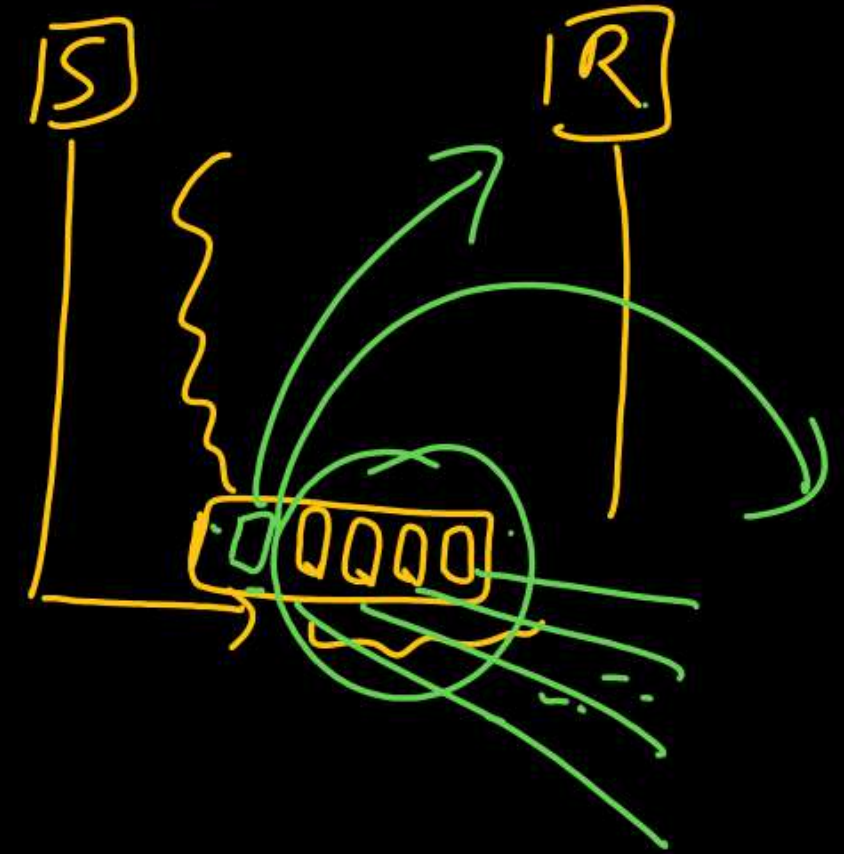
- **Reserved:** This is a 6-bit field reserved for future use.

➤ **Control**: defines 6 different control bits or flags. One or more of these bits can be set at a time.



Flag Description:

- URG** The value of the urgent pointer field is valid.
- ACK** The value of the acknowledgment field is valid.
- PSH** Push the data.
- RST** Reset the connection.
- SYN** Synchronize sequence numbers during connection.
- FIN** Terminate the connection.



➤ **Window size:**

- 16 bits defines size of the receiving window (rwnd), in bytes, that the other party must maintain.
- Maximum size of the window is 65,535 bytes.
- This value is determined by the receiver. The sender must obey dictation of the receiver.

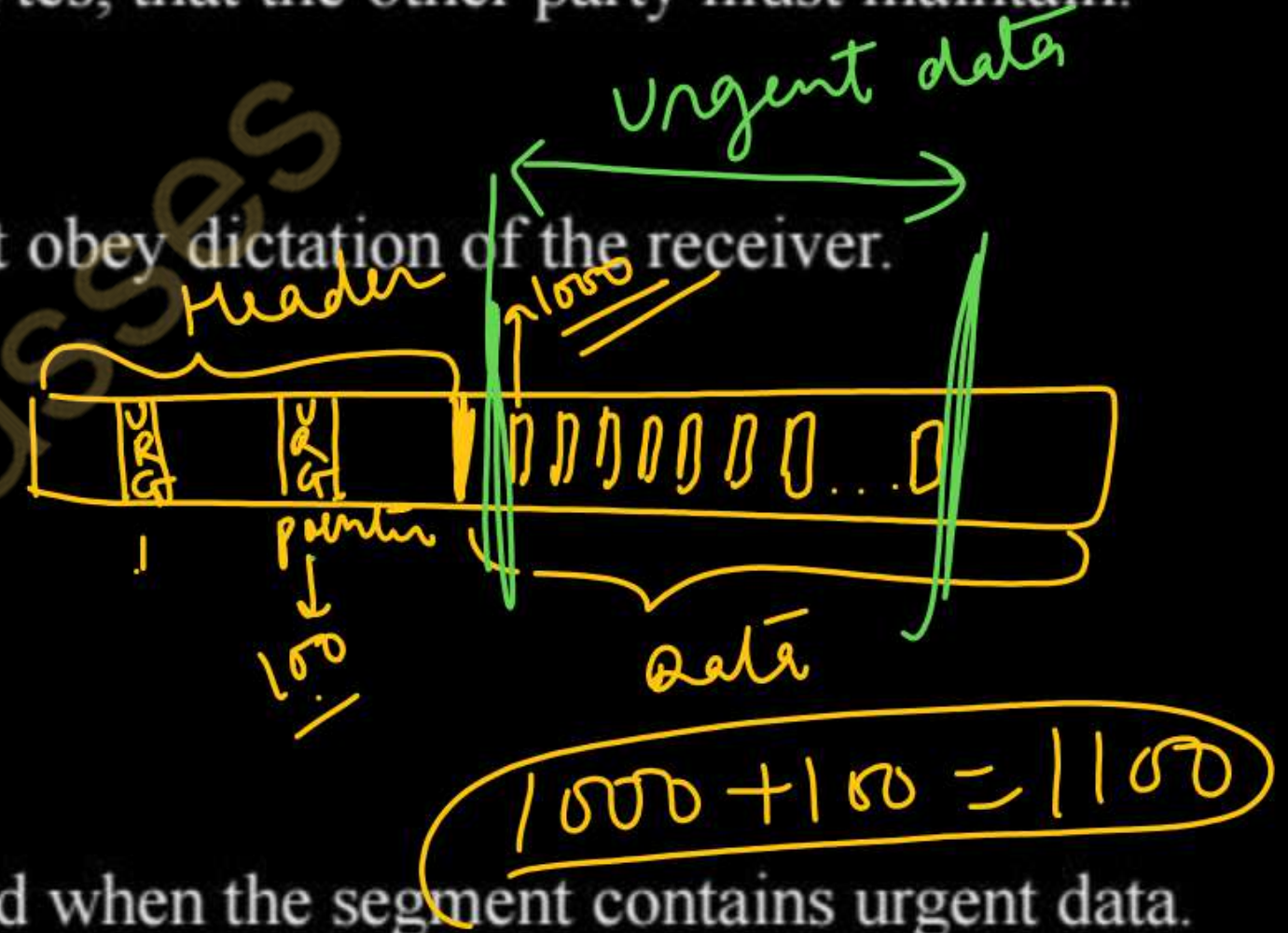
➤ **Checksum:**

- 16-bit field contains checksum.
- Inclusion of the checksum for TCP is mandatory.

➤ **Urgent pointer:** ✓✓

- 16-bit field, which is valid only if urgent flag is set, is used when the segment contains urgent data.
- It defines the number that must be added to the sequence number to obtain the number of the last urgent byte in the data section of the segment.

➤ **Options:** There can be up to 40 bytes of optional information in the TCP header.



Sunder

window size = 10000 bytes

MSS = 1000 byte ✓

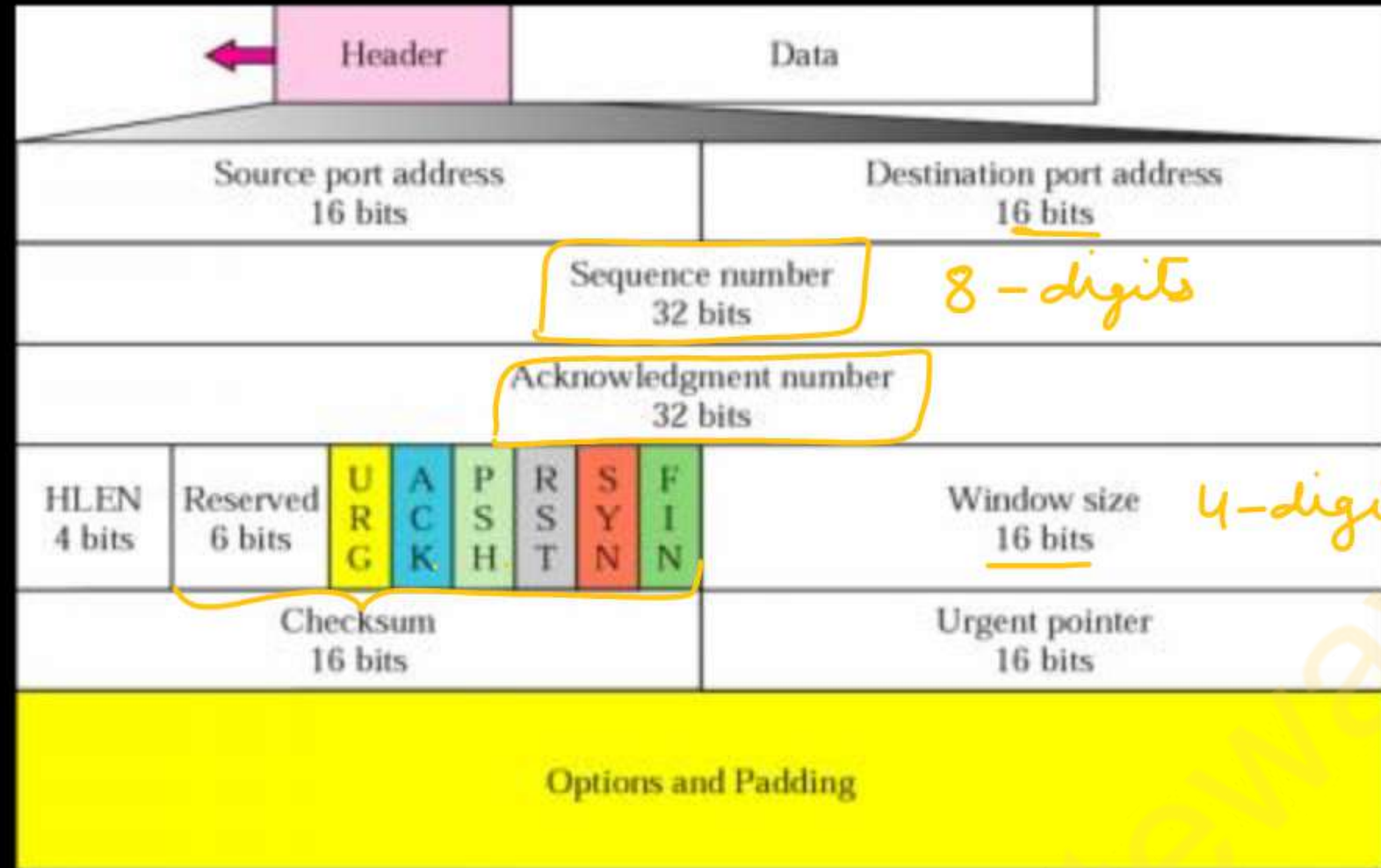
Q: Following is a dump of a TCP header in hexadecimal format

(AKTU)

05320017 00000001 00000000 500207FF 00000000

- 1) What is the source port number?
- 2) What is the destination port number?
- 3) What is sequence number?
- 4) What is the acknowledgement number?
- 5) What is the length of the header? $\Rightarrow$ 20 byte
- 6) What is the window size?
- 7) Are any options present in the header $\Rightarrow$ X

053200170000000100000000500207FF00000000



- 1) Source port number:- (2 byte) -> 0532
- 2) Destination port number:- (2 byte) -> 0017
- 3) Sequence number:- 000000001 -> 1
- 4) Acknowledgement number:- 00000000 -> 0
- 5) Length of the header (4 bits) -> 5
- 6) Window size -> 07FF

AKTU PYQs

1. Write about Registered ports (AKTU 2023-24)
2. What is difference between network layer delivery and transport layer delivery. (AKTU 2018-19)
3. Enumerate on TCP header and working of TCP and differentiate TCP and UDP with frame format. (AKTU 2016-17,2017-18,2018-19)
4. Differentiate between connection less and connection oriented (AKTU 2023-24)
5. Define TCP features and header in detail (AKTU 2023-24)
6. List prime three functionality of transport layer. (AKTU 2021-22)
7. Explain the use of RST flag in TCP header (AKTU 2021-22)
8. How does transport layer perform duplication control? (AKTU 2017-18)

The following is the dump of a TCP header in hexa decimal format:

05320017 00000001 00000000 500207FF 00000000

- (i) What is the sequence number?
- (ii) What is the destination port number?
- (iii) What is the acknowledgment number?
- (iv) What is the window size?



AKTU

B.Tech 6th Semester
CS IT & Allied



Computer Networks

Unit-4: Lecture 2

Today's Target

- Multiplexing
- TCP Connection Management
- RST flag
- URG Flag and Urgent pointer
- PSH Flag
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified



(Port-Addressing)

Multiplexing and Demultiplexing

The addressing mechanism allows multiplexing and demultiplexing by the transport layer.

Multiplexing:

- At the sender site, there may be several processes that need to send packets.
- The transport layer protocol accepts messages from different processes, differentiated by their assigned port numbers.
- After adding the header, the transport layer passes the packet to the network layer.

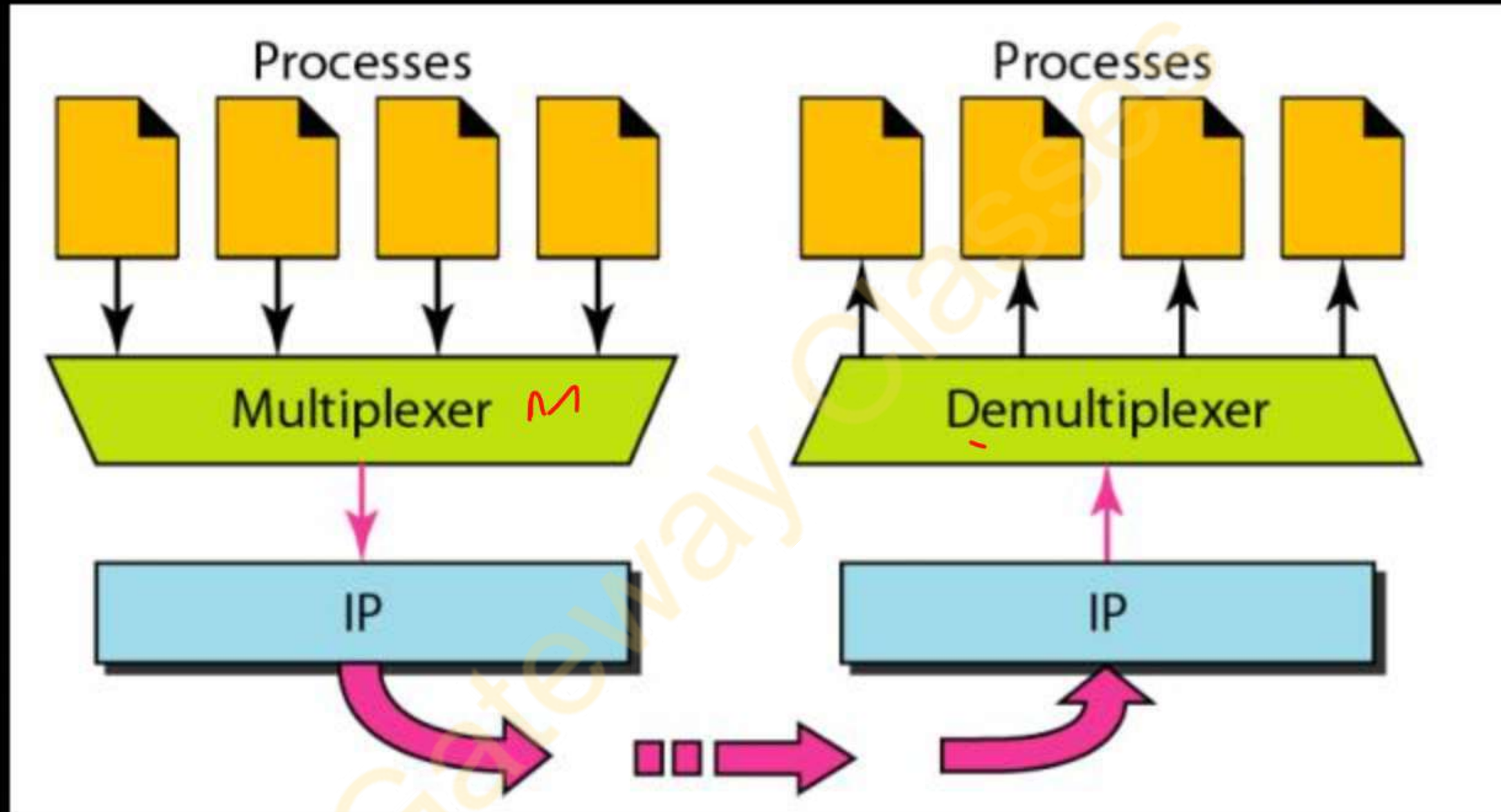
Demultiplexing:

- At the receiver site, the relationship is one-to-many and requires demultiplexing.
- The transport layer receives datagrams from the network layer.
- After error checking and dropping of the header, the transport layer delivers each message to the appropriate process based on the port number.

Multiplexing and Demultiplexing

Sender

Receiver

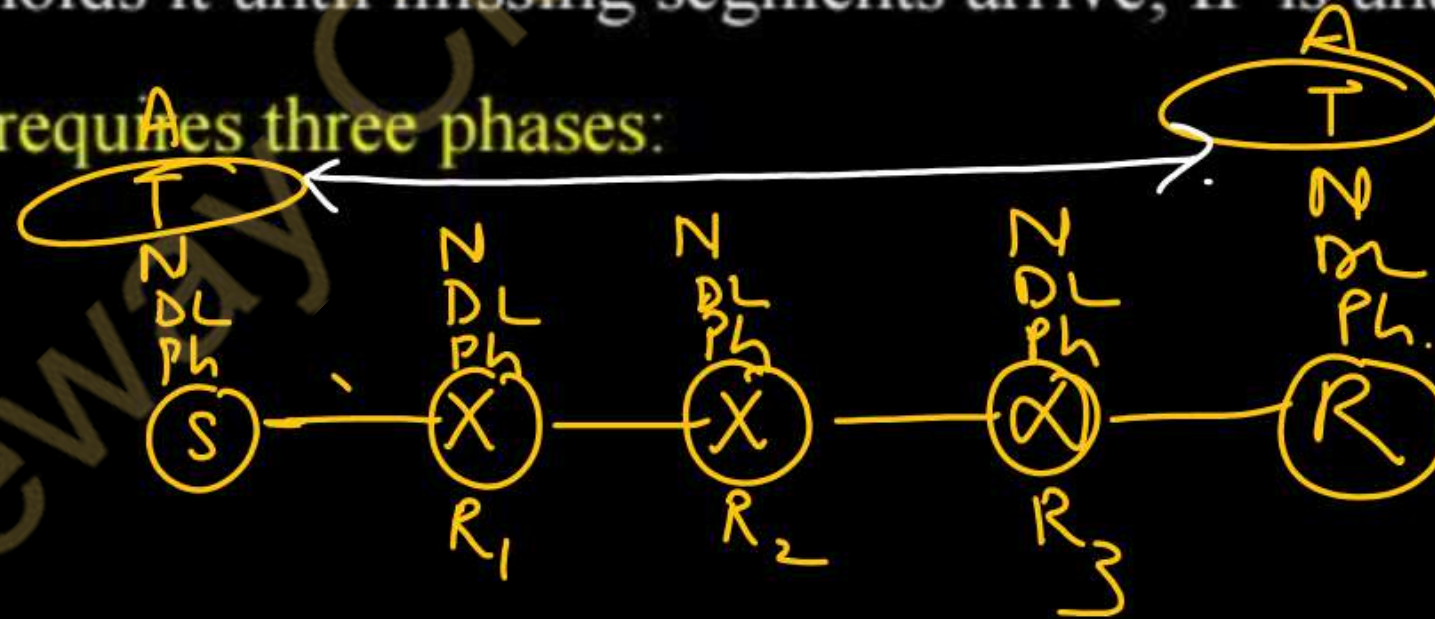


A TCP Connection

- TCP is connection-oriented as it establishes a virtual path between the source and destination.
- All segments belonging to a message are then sent over this virtual path.
- TCP uses services of IP to deliver individual segments to receiver, but it controls the connection itself.
- If a segment is lost/corrupted, it is retransmitted. IP is unaware of this retransmission.
- If a segment arrives out of order; TCP holds it until missing segments arrive; IP is unaware of this too.

In TCP, connection-oriented transmission requires three phases:

1. connection establishment
2. data transfer
3. connection termination



Connection Establishment

TCP transmits data in full-duplex mode.

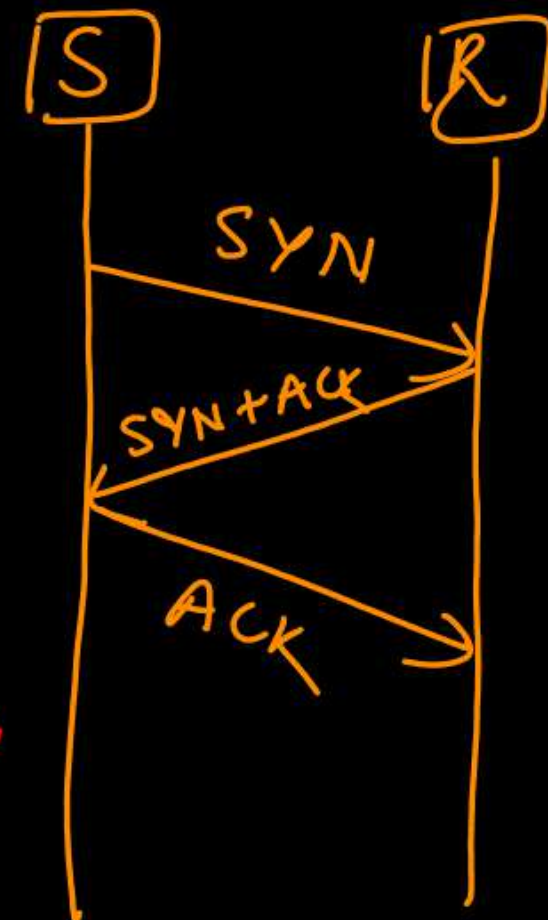
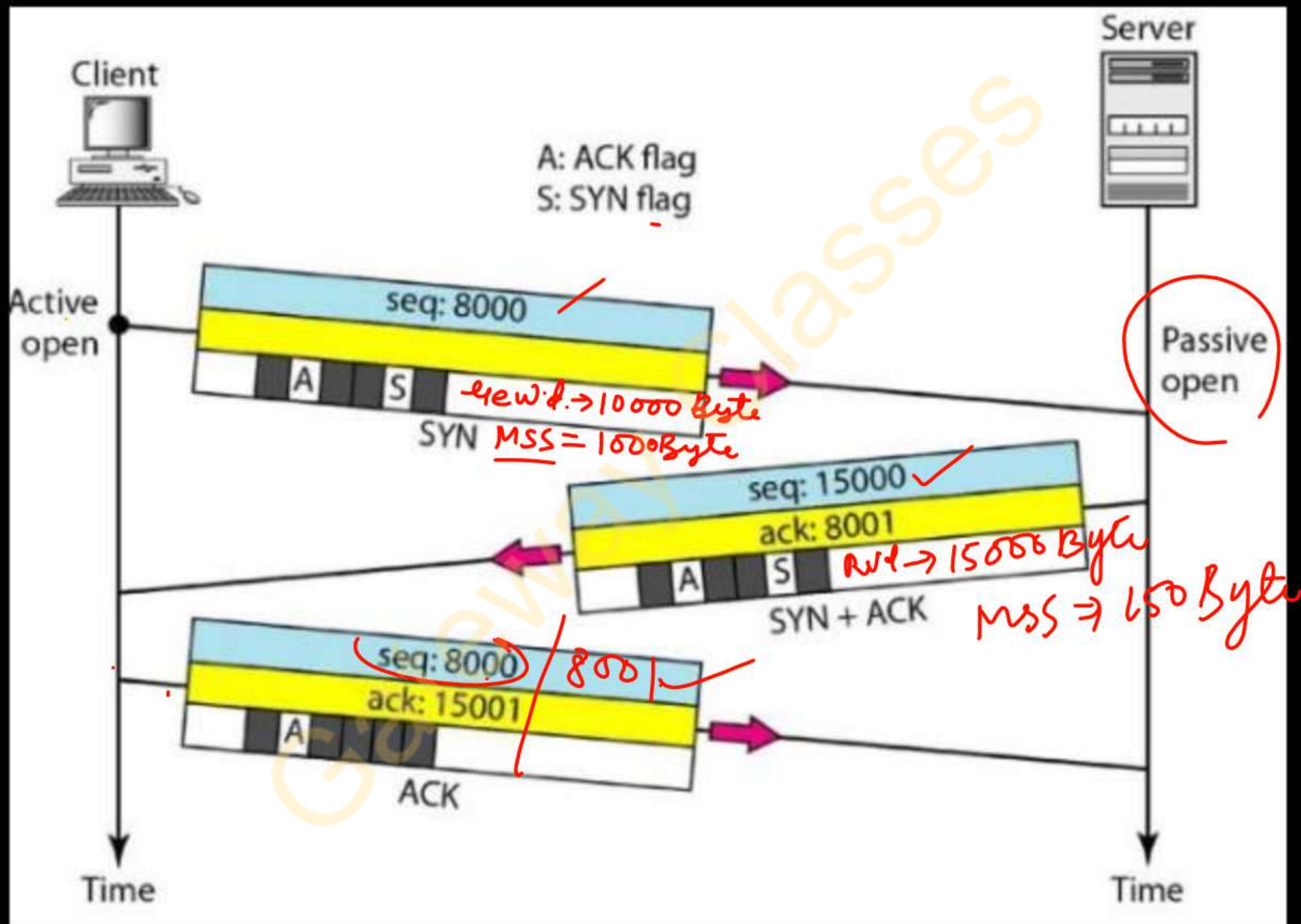
When two TCPs in two machines are connected, they are able to send segments to each other simultaneously.

Three-Way Handshaking:

Imp

- The connection establishment in TCP is called three way handshaking.
- Suppose client, wants to make a connection with the server, using TCP as the transport layer protocol.
- The process starts with the server. The server program tells its TCP that it is ready to accept a connection. This is called a request for a passive open.
- The client program issues a request for an *active open*. A client that wishes to connect to an open server tells its TCP that it needs to be connected to that particular server.
- TCP can now start the three-way handshaking process .

Connection establishment using three-way handshaking



The three steps in this phase are as follows.

Step-1: The client sends the first segment, a SYN segment, in which only the SYN flag is set.

- This segment is for synchronization of sequence numbers.
- It consumes one sequence number.
- When the data transfer starts, the sequence number is incremented by 1.

A SYN segment cannot carry data but it consumes one sequence number.

Step-2: The server sends second segment, a SYN+ACK segment, with 2 flag bits set: SYN and ACK.

- It is a SYN segment for communication in other direction and serves as the ack for the SYN.
- It consumes one sequence number.

SYN +ACK segment cannot carry data, but consume one sequence number.

Step-3:

- The client sends an ACK segment to acknowledge the receipt of second segment with ACK flag and ack number field.
- Sequence number in this segment is the same as the one in the SYN segment.
- ACK segment does not consume any sequence numbers.

✓ An ACK segment, if carrying no data, consumes no sequence number.

Simultaneous Open: ✓

- This rare situation may occur when both processes issue an active open.
- Both TCPs transmit a SYN + ACK segment to each other, and one single connection is established between them.

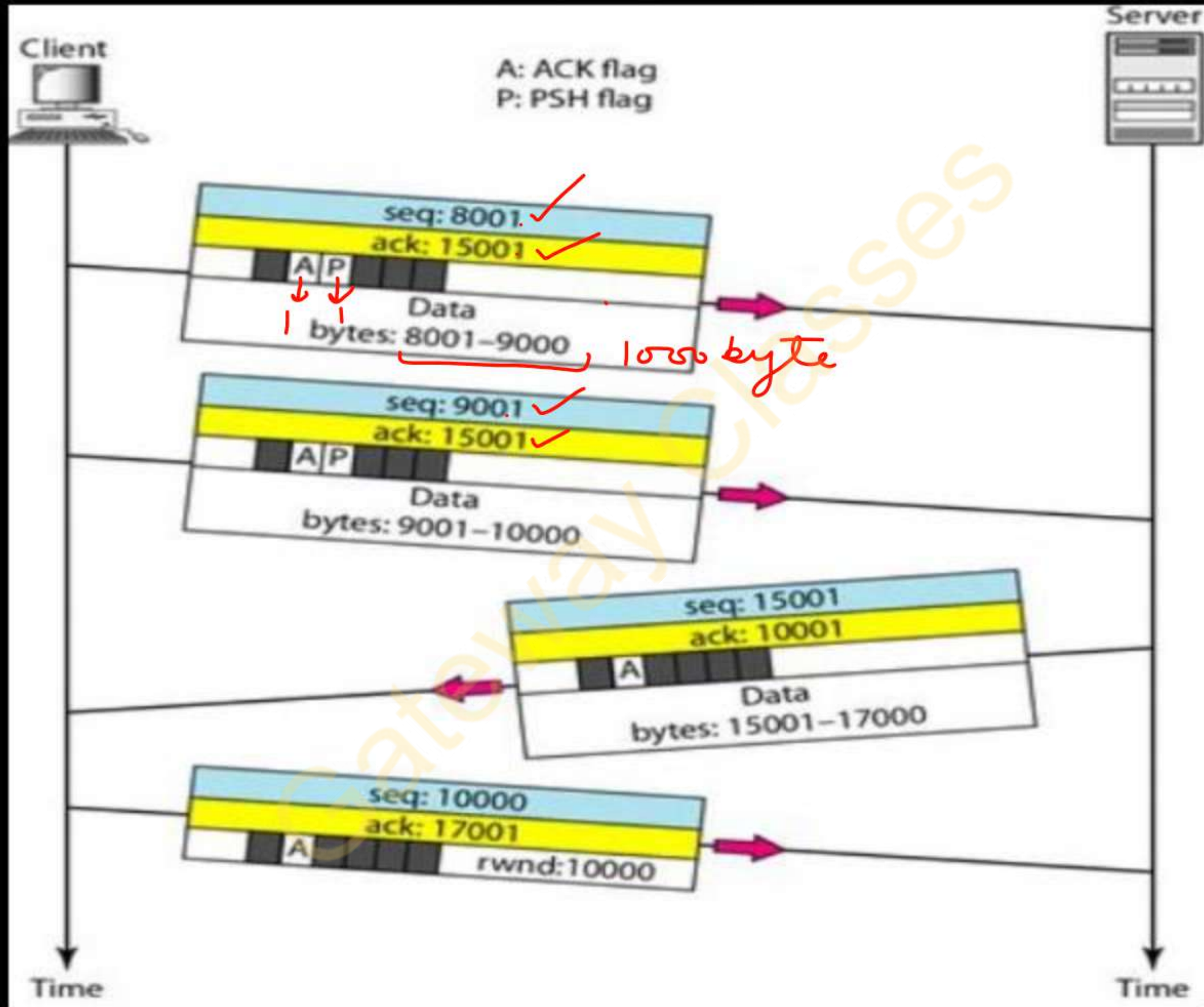
SYN Flooding Attack:

- A malicious attacker sends a large number of SYN segments to a server, pretending that each of them is coming from a different client by faking the source IP addresses in the datagrams.
- Server allocates the necessary resources and sends the SYN +ACK segments to the fake clients, which are lost.
- If the number of SYN segments is large, server runs out of resources and may crash.
- This is a denial-of-service attack as server denies any further service to every forthcoming request.

Preventive Strategies:

- Imposing a limit on connection requests during a specified period of time.
- Filter out datagrams coming from unwanted source addresses.
- Postpone resource allocation until the entire connection is set up.

Data Transfer



Data Transfer

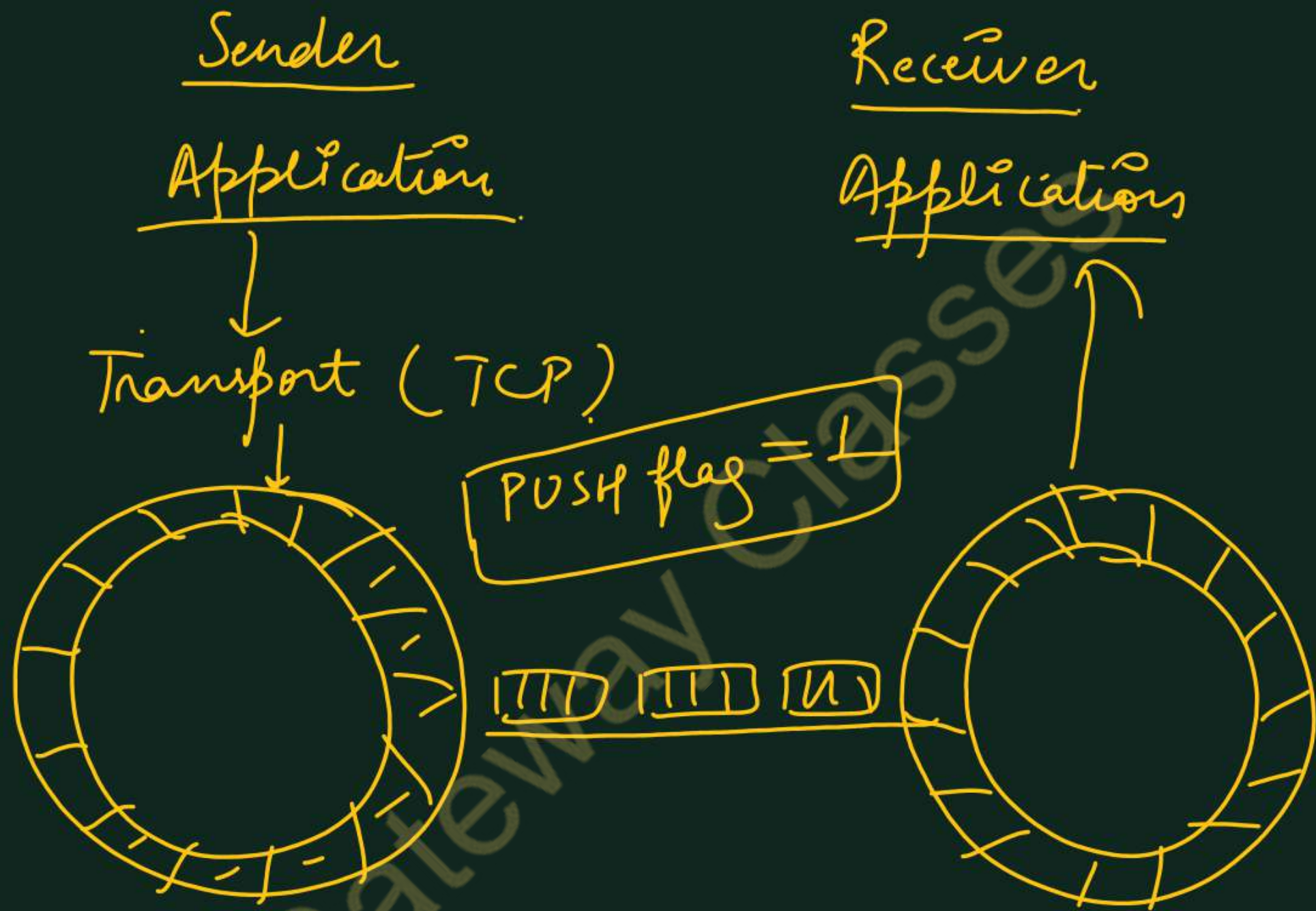
- After connection is established, bidirectional data transfer can take place.
- The client and server can both send data and acknowledgments.
- The acknowledgment is piggybacked with the data.
- In example, client sends 2000 bytes of data in two segments. Server then sends 2000 bytes in one segment.
- The client sends one more segment.
- The first three segments carry both data and ack, but the last segment carries only an ack as there
- are no more data to be sent.

Pushing Data:

Sending and receiving TCP uses a buffer to store the stream of data to be sent by sending application and to be delivered to receiving application program whenever it is convenient or ready to receive.

Example:

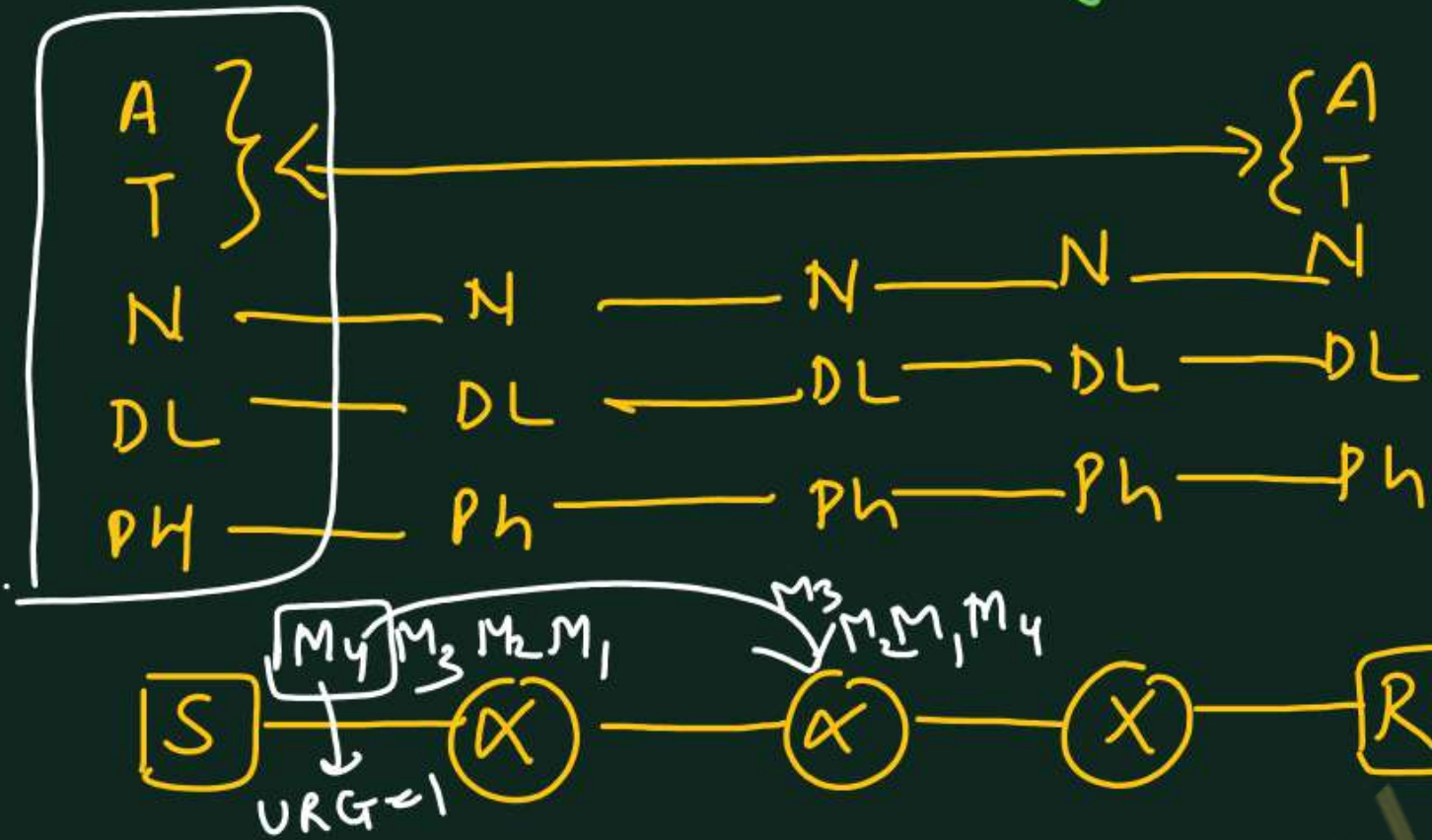
- Application program on one site wants to send a keystroke to the application at the other site and receive an immediate response. Delayed transmission and delayed delivery may not be acceptable.
- Then application program at the sending site can request a *push* operation.
- This means that the sending TCP must not wait for the window to be filled.
- It must create a segment and send it immediately.
- The sending TCP must also set the push bit (PSH) to let the receiving TCP know that the segment includes data that must be delivered to the receiving application program as soon as possible and not to wait for more data to come.



Urgent Data: (Urgent pointer)

- Data are presented from the application program to TCP as a stream of bytes.
- Suppose sending application program wants a piece of data to be read out of order urgently by the receiving application, but normally it will be delivered to the receiving application program after all the data have been processed sent before this.
- The solution is to send a segment with the URG bit set.
- The sending TCP creates a segment and inserts the urgent data at the beginning of the segment.
- Rest of the segment can contain normal data from the buffer.
- The urgent pointer field in the header defines the end of the urgent data and the start of normal data.
- When receiving TCP receives a segment with URG bit set, it extracts urgent data from segment, using the value of the urgent pointer, and delivers them, out of order, to the receiving application program.

At N/w layer, this urgent data segment is given priority ⑦

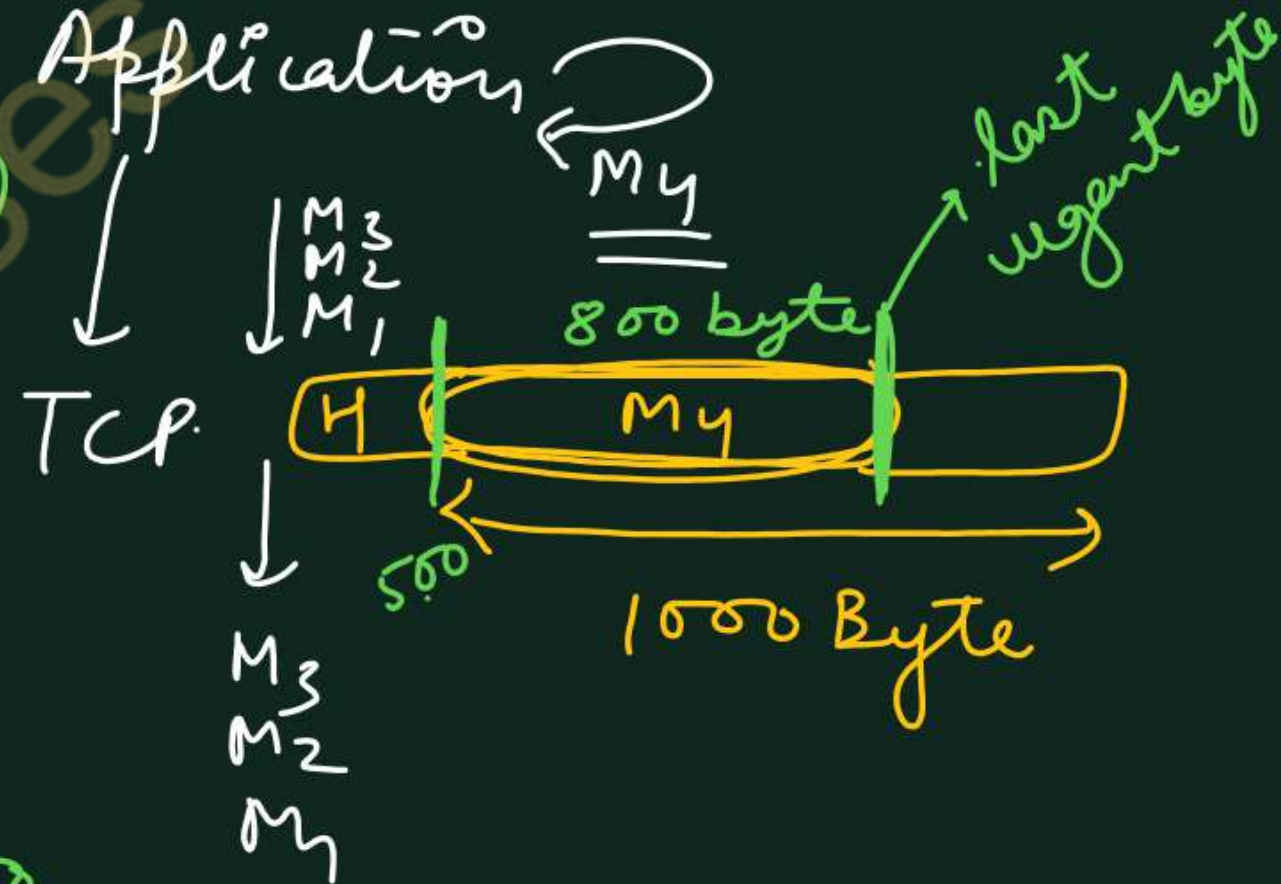


URG Flag $\Rightarrow$ 1/0.

Urgent pointer = 800

Seq No of First byte = 500

$500 + 800 = 1300 \Rightarrow$ Seq. no of last urgent byte



Connection Termination

- Any of the two parties (client or server) can close the connection, although usually initiated by client.

Two options for connection termination:

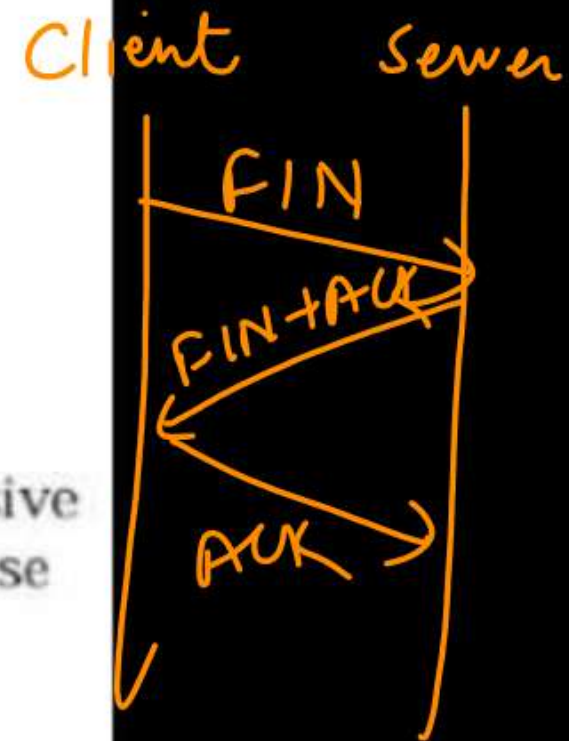
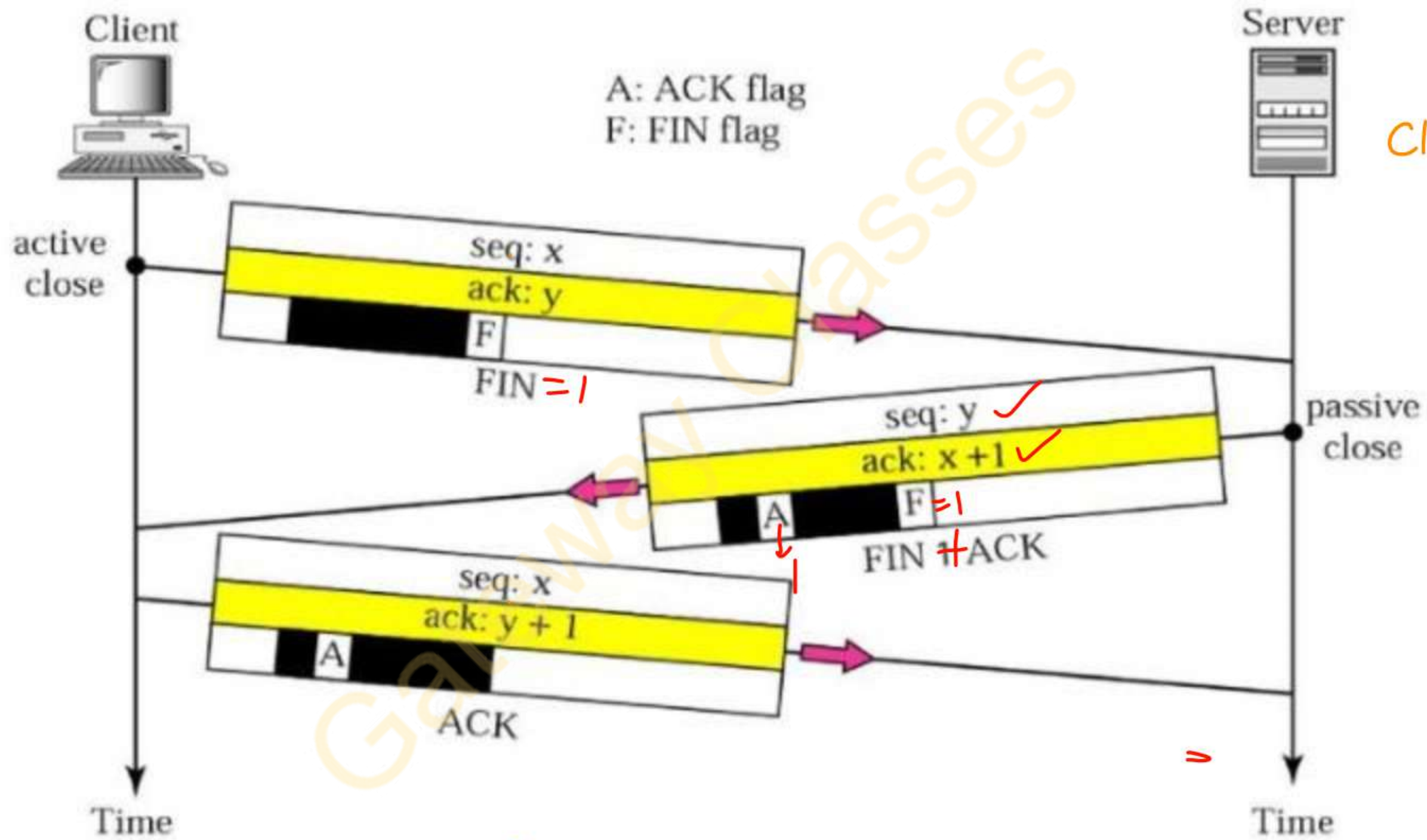
- Three-way handshaking
- Four-way handshaking with a half-close option

Three-Way Handshaking:

Step-1:

- Client TCP, after receiving a close command from client process, sends a FIN segment in which the FIN flag is set.
- FIN segment can include last chunk of data sent by the client, or it can be just a control segment.
- If it is only a control segment(does not carry data), it consumes only one sequence number.

Connection Termination with Three-way handshaking



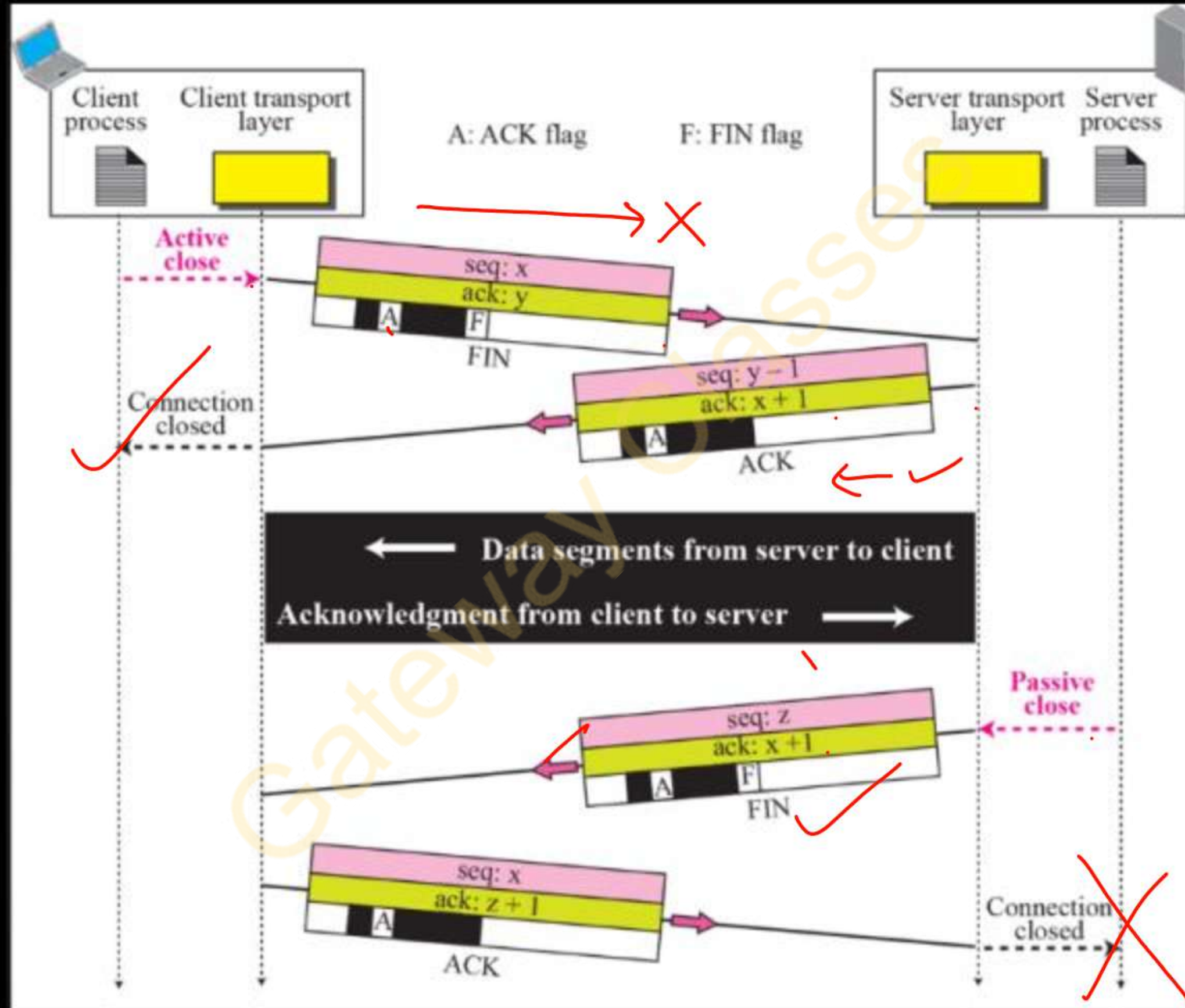
Step-2:

- Server TCP, after receiving the FIN segment, informs its process of the situation and sends FIN +ACK segment, to confirm the receipt of the FIN segment from the client and at the same time to announce the closing of the connection in the other direction.
- This segment can also contain the last chunk of data from the server.
- If it does not carry data, it consumes only one sequence number.

Step-3:

- Client TCP sends an ACK segment, to confirm the receipt of the FIN segment from the TCP server. This segment contains the ack number(1 plus seq number received in FIN segment) from server.
- This segment cannot carry data and consumes no sequence numbers.

Connection Termination with Four-way handshaking with a half-close option



Half-Close: (normally initiated by the client)

- In TCP, one end can stop sending data while still receiving data. This is called a half-close.
- It can occur when the server needs all the data before processing can begin.
- Example: When client sends data to server to be sorted, the server needs to receive all data before sorting can start. Client, after sending all data, can close the connection in the outbound direction and remain open to receive the sorted data. The server outbound direction must remain open.
- Client half-closes the connection by sending a FIN segment.
- Server accepts half-close by sending ACK segment.
- Data transfer from client to server stops. The server, however, can still send data.
- When server has sent all processed data, it sends a FIN segment, which is acknowledged by client.
- After half-closing of the connection, data can travel from the server to client and acknowledgments can travel from the client to the server. The client cannot send any more data to the server.

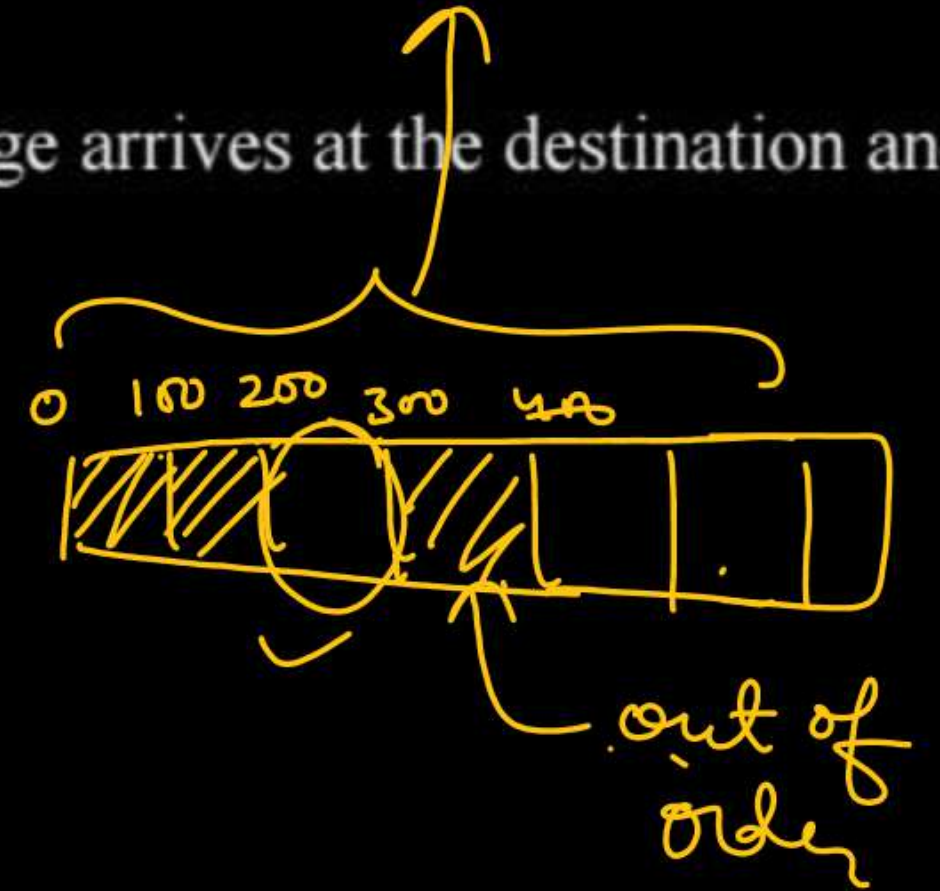
Q: Explain the use of RST flag in TCP header (AKTU 2021-22)

RST Flag:

- The TCP RST flag indicates that connection should be immediately terminated.
 - If in case, the connection has already been closed from one side (side A) and the other side (side B) suddenly comes back and wishes to continue the session for data transfer, host (side A) usually responds by sending a RST packet back to the same connection and stating that no longer interested in data transfer. RST bit will be set to high in the TCP header flag.
1. The packet is an initial SYN packet trying to establish a connection to a server port on which no process is listening.
 2. The packet arrives on a TCP connection that was previously established but the local application has already closed its socket or exited and the OS has closed the socket.
 3. Other circumstances such as attempts to hijack a TCP connection.

AKTU PYQs

1. Explain the three-way handshaking protocol to establish the transport level connection (AKTU 20216-17,2017-18)
2. Enumerate how the transport layer ensure that the complete message arrives at the destination and in the proper order. (AKTU 2016-17, 2017-18)





AKTU

B.Tech 6th Semester
CS IT & Allied



Computer Networks

Unit-4: Lecture 3

Today's Target

- TCP Flow Control
- TCP Error Control
- TCP Congestion Control
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified



Flow Control

TCP uses a sliding window protocol between the *Go-Back-N* and Selective Repeat.

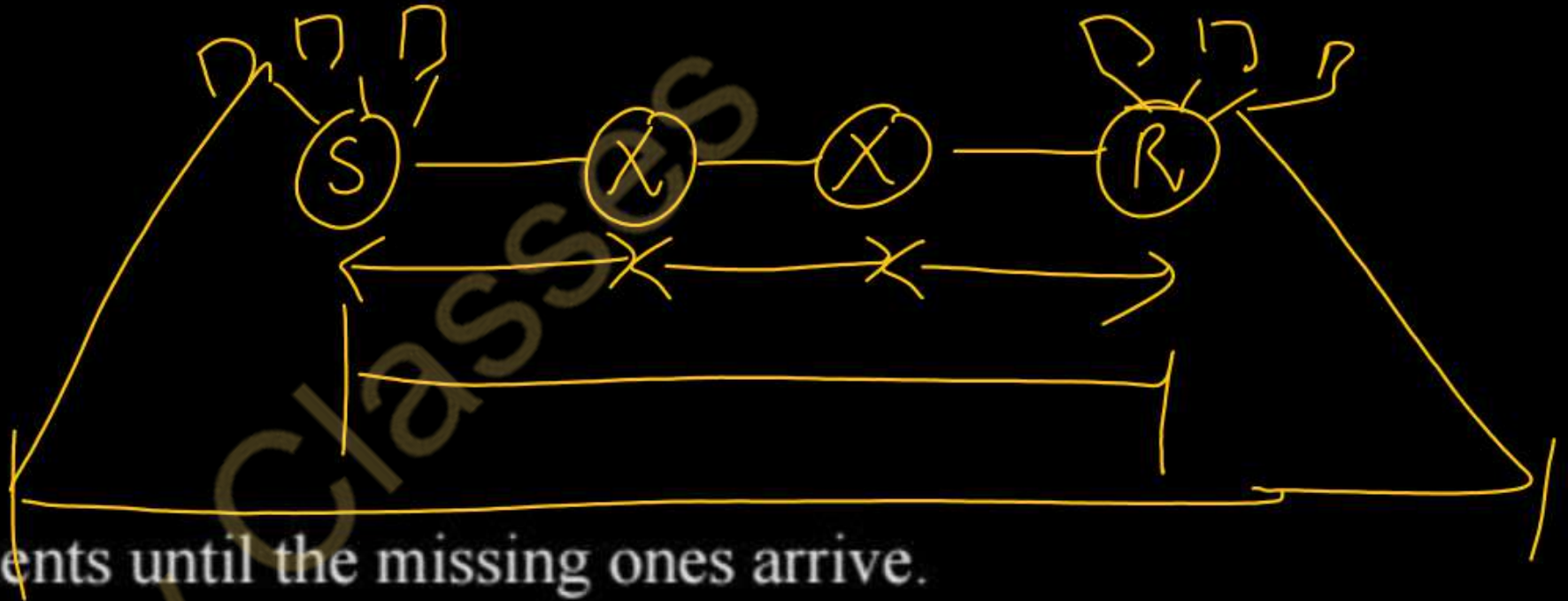
➤ Why like Go-Back-N:

- Does not use NAKs
- Sends cumulative acks

➤ Why like Selective Repeat:

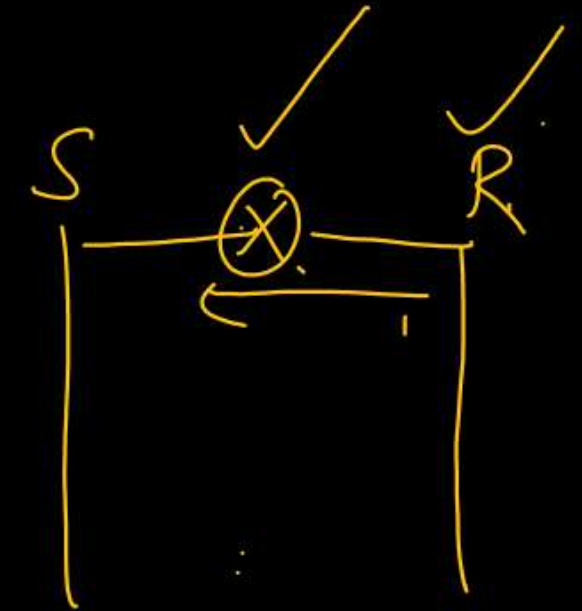
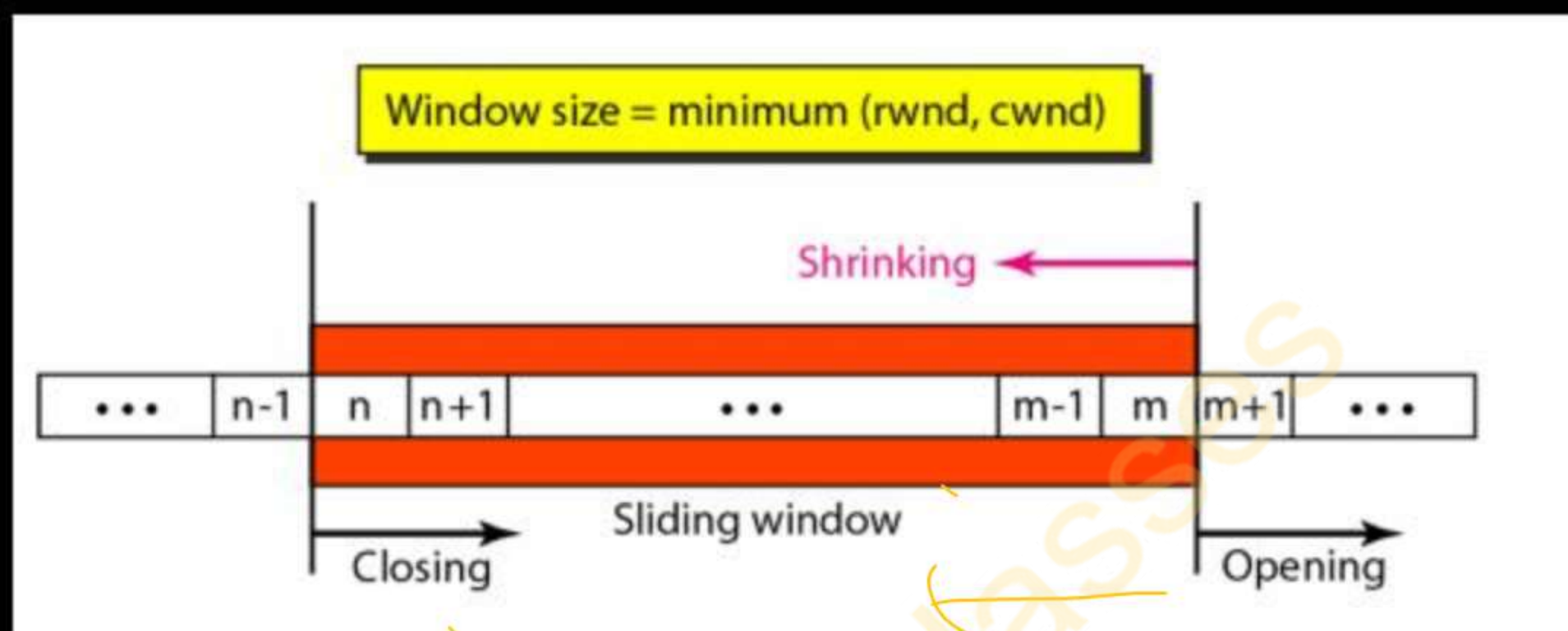
- Receiver holds the out-of-order segments until the missing ones arrive.
- Sender window size = receiver window size ✓

(Setup during connection establishment. Both sender and receiver advertise their windows)



Differences between TCP sliding window and sliding window at data link layer:

1. Sliding window of TCP is byte-oriented; while in data link layer it is frame-oriented.
2. TCP's sliding window is of variable size; while in data link layer was of fixed size.



- Bytes inside the window can be sent without worrying about ack.
- The imaginary window has two walls: one left and one right.
- The window is *opened, closed, or shrunk*. These three activities are in the control of receiver (and depend on congestion in the network), not the sender.
- The sender must obey the commands of the receiver in this matter.
- **Opening a window** means moving right wall to the right (more new bytes in buffer eligible for sending).
- **Closing the window** means moving left wall to right (some bytes have been acknowledged.)

- **Shrinking the window** means moving the right wall to the left.
- This is a problem if the sender has already sent these bytes.
- TCP sliding windows are byte-oriented.

Size of window at one end = $\text{MIN}[\text{receiver window (rwnd)}, \text{congestion window (cwnd)}]$

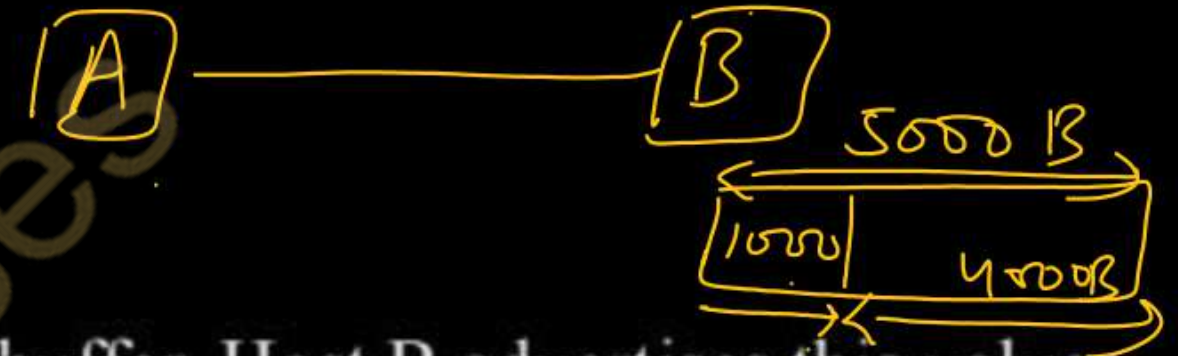
- *receiver window* is the value advertised by the opposite end in a segment containing ack.
- It is the number of bytes the other end can accept before its buffer overflows and data are discarded.
- *congestion window* is a value determined by the network to avoid congestion.

Q: What is the value of the receiver window (*rwnd*) for host A if the receiver, host B, has a buffer size of 5000 bytes and 1000 bytes of received and unprocessed data?

Solution

The value of $rwnd = 5000 - 1000 = 4000$.

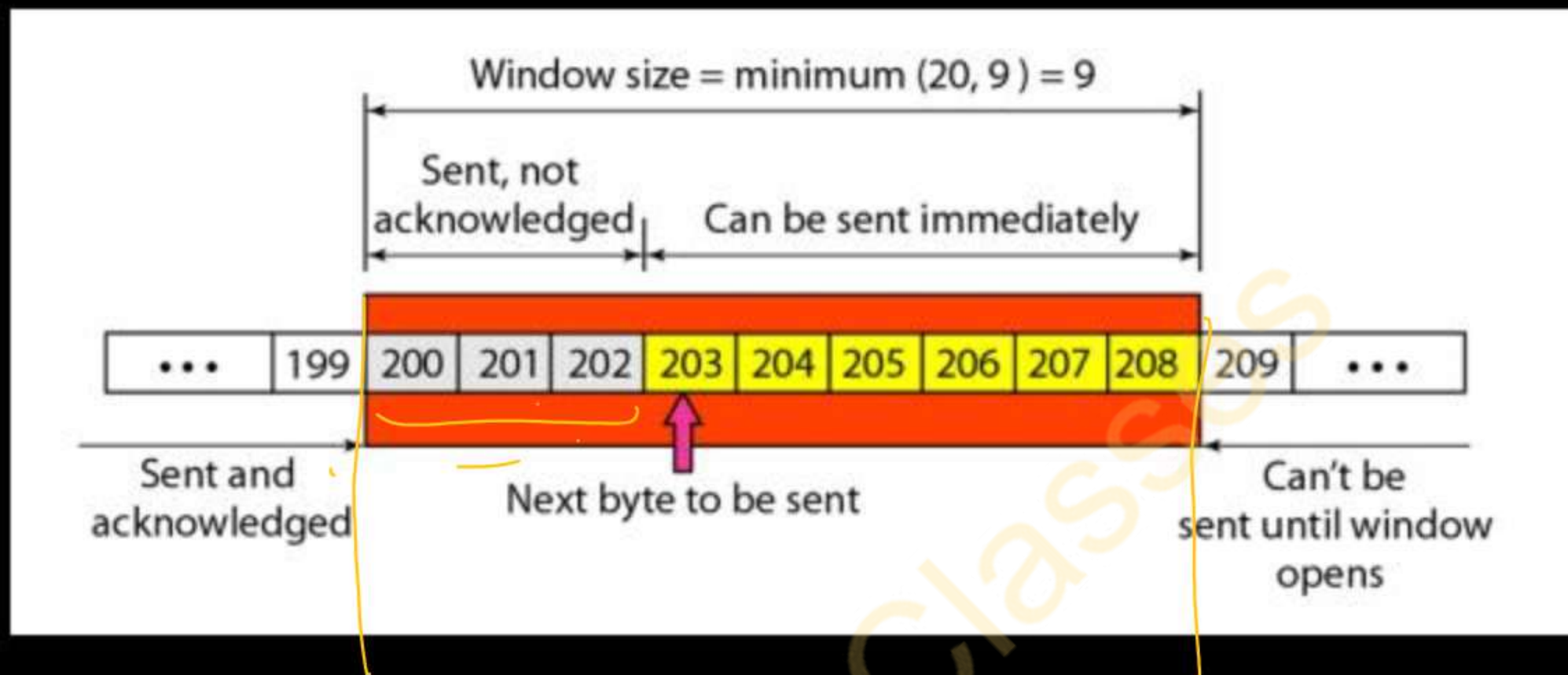
Host B can receive only 4000 bytes of data before overflowing its buffer. Host B advertises this value in its next segment to A.



Q: What is the size of the window for host A if the value of *rwnd* is 3000 bytes and the value of *cwnd* is 3500 bytes?

Solution

The size of the window is the smaller of *rwnd* and *cwnd*, which is 3000 bytes.



- The sender has sent bytes up to 202. We assume that *cwnd* is 20 (in reality thousands of bytes).
- The receiver has sent an ack number of 200 with *rwnd* of 9 bytes.
- The size of the sender window is the minimum of *rwnd* and *cwnd*, or 9 bytes.
- Bytes 200 to 202 are sent, but not acknowledged.
- Bytes 203 to 208 can be sent without worrying about acknowledgment.
- Bytes 209 and above cannot be sent.

Flow control is maintained by following:

1. Advertisement window size

2. Persistent Timer

- Receiver informs sender about amount of data it can currently buffer using rwnd field in TCP header.
- It helps sender limit its transmission rate, ensuring it doesn't exceed the receiver's capacity. TCP uses a sliding window protocol where the sender can transmit multiple segments before receiving an ack.
- When receiver acknowledges a segment, it also updates the receive window, reflecting amount of data it can now buffer.

$W_S = 3000 \text{ Bytes}$ [S] [R] $W_R = 2000 \text{ Bytes}$

$W_1 = 2000 \text{ B}$

Request, $W_S = 3000 \text{ Byte}$

Reply, $W_R = 2000 \text{ Byte}$

ACK

Data (500 B)

Data (500 B)

ACK $W_2 = 1000 \text{ B}$

Data (500 B)

Data (500 B)

ACK $W_2 = 0 \text{ B}$

$W_2 = 1400 \text{ B}$

1 Byte Probe

$W_2 = 2000 \text{ Byte}$

$W_2 = 2000 \text{ B}$

$W_2 = 1000 \text{ B}$

1000 B

$W_2 = 0 \text{ B}$

$W_2 = 2000 \text{ B}$

Sender still has data to be sent

Stop Sending

Persistent Timer

Timeout

unprocessed data
(Not yet delivered to Application)

↓ suppose 1400 B not processed.

Error Control:

- TCP provides reliability using error control.
- Error control includes mech. for detecting corrupted segments, lost segments, out-of-order segments, and duplicated segments and for correcting errors..

Error detection and correction in TCP is achieved through checksum, acknowledgment, and time-out.

Checksum:

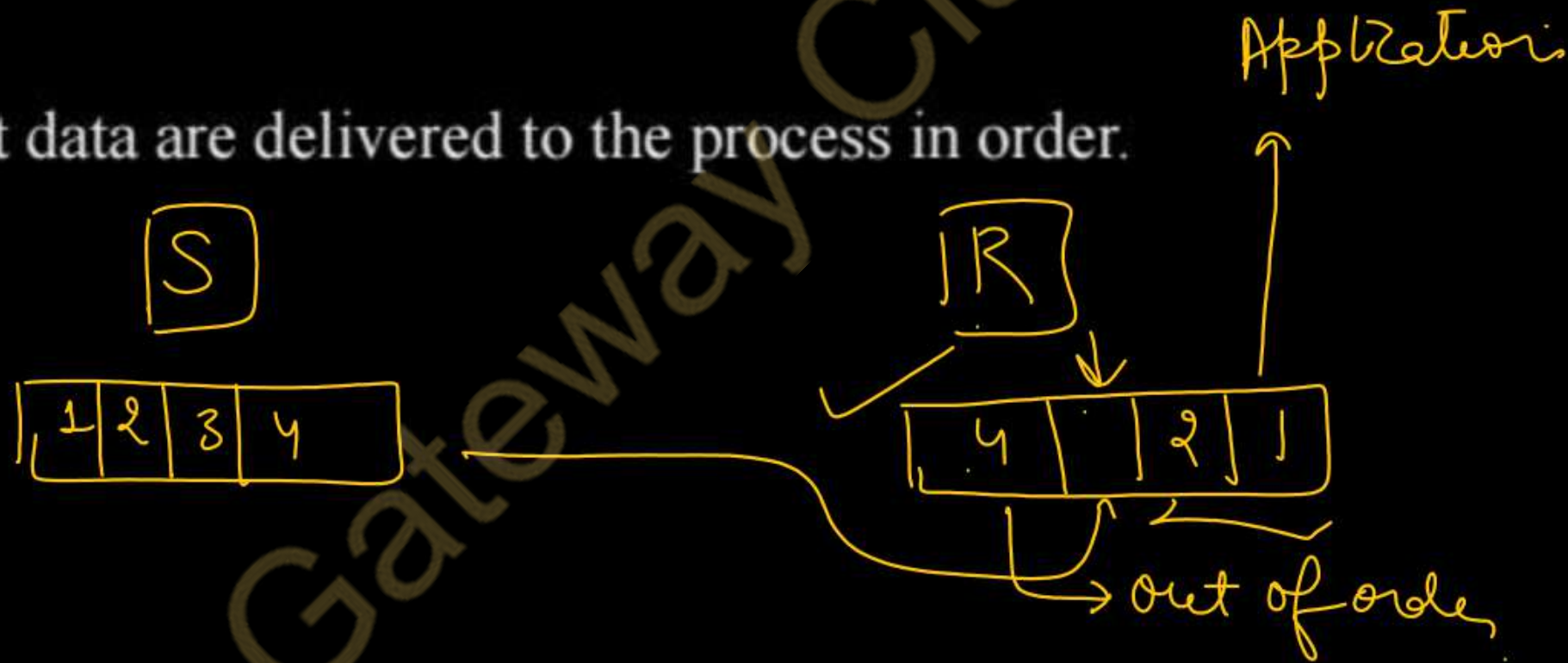
- Each segment includes a 16-bit checksum field to check for a corrupted segment.
- If the segment is corrupted, it is discarded by the destination TCP and is considered as lost.

Acknowledgment:

- TCP uses acks to confirm receipt of data segments.
- Control segments that carry no data but consume a sequence number are also acknowledged.
- ACK segments are never acknowledged.
- ACK segments do not consume sequence numbers and are not acknowledged.

Out-of-Order Segments

- When a segment is delayed, lost, or discarded, the segments following that segment arrive out of order. Most implementations of TCP do not discard the out-of-order segments.
- They store them temporarily and flag them as out-of-order segments until the missing segment arrives.
- TCP guarantees that data are delivered to the process in order.



Retransmission:

- Error control mechanism is achieved through retransmission of segments.
- When a segment is corrupted, lost, or delayed, it is retransmitted. A segment is retransmitted on two occasions:
 - ✓ when a retransmission timer expires. ✓
 - OR
 - ✓ when the sender receives three duplicate ACKs.
- No retransmission occurs for segments that do not consume sequence numbers.
- No retransmission timer is set for an ACK segment.



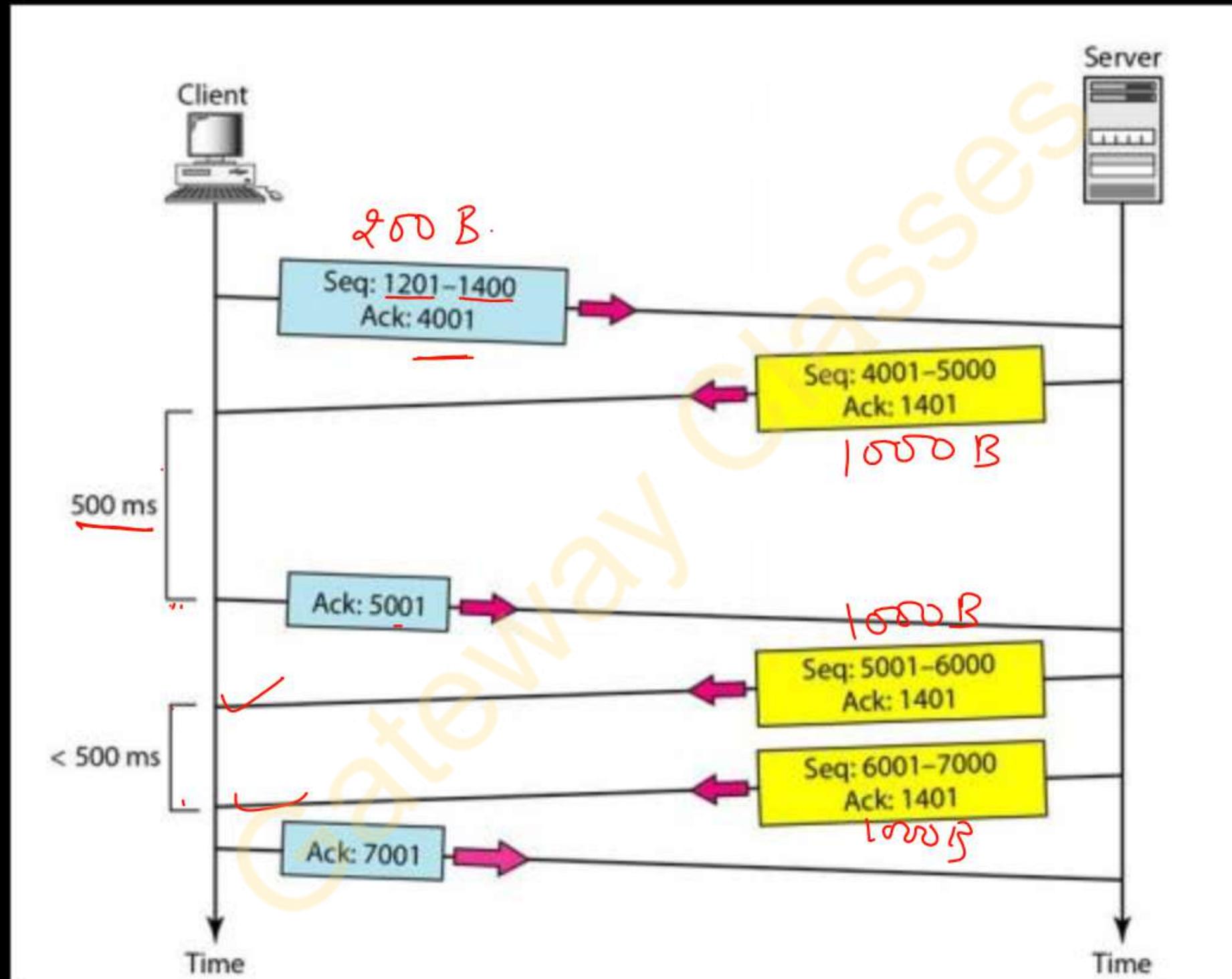
Retransmission After RTO → Retransmission Timeout

- TCP maintains one retransmission time-out timer for all sent, but not acknowledged segments.
- When timer matures, the earliest outstanding segment is retransmitted even though lack of a received ACK can be due to a delayed segment, a delayed ACK, or a lost acknowledgment.
- **No time-out timer is set for a segment that carries only an ack.**
- The value of RTO is updated based on the round-trip time (RTT) of segments.

Retransmission After Three Duplicate ACK Segments(fast retransmission) :

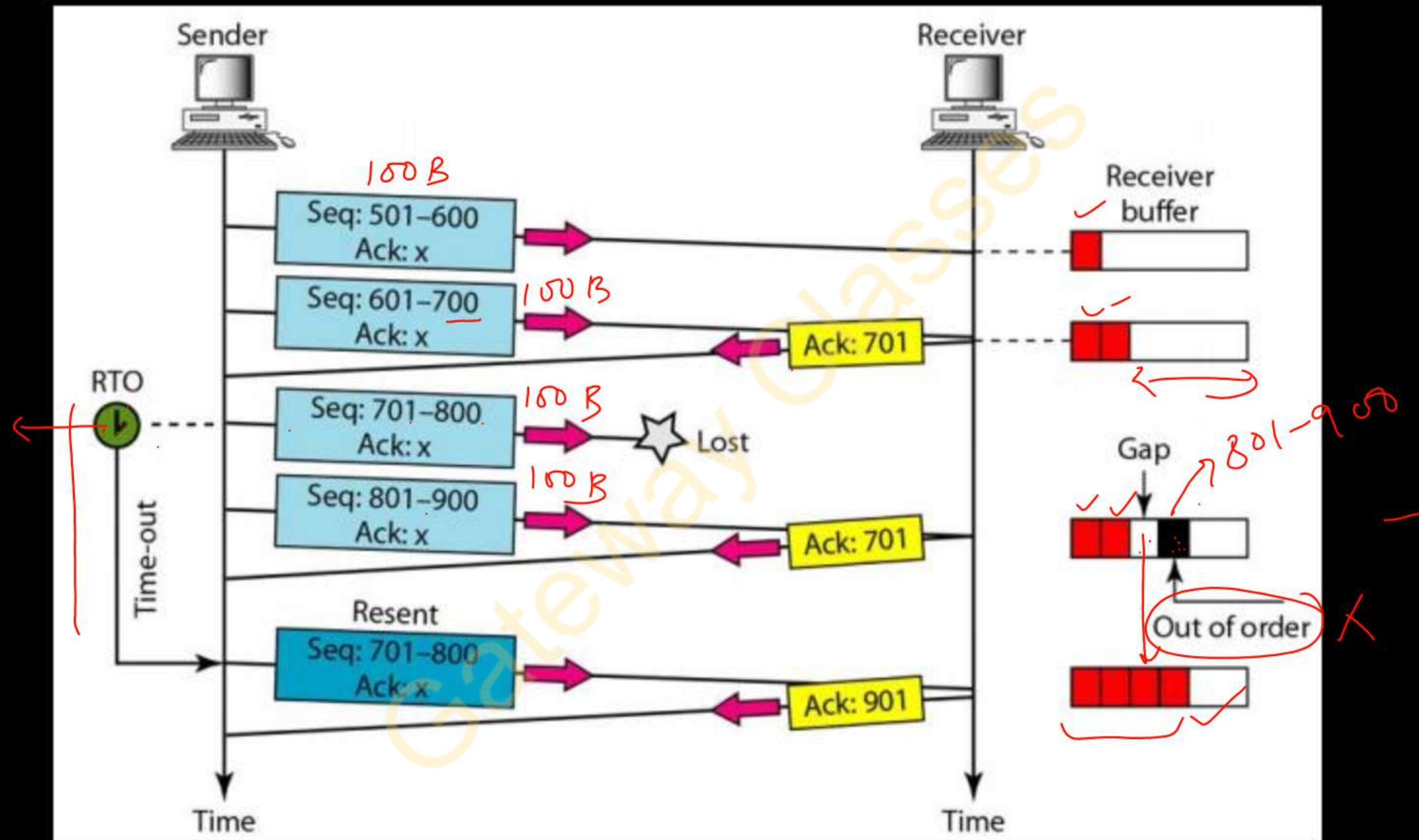
- If RTO is very large, one segment is lost and receiver receives so many out-of-order segments that they cannot be saved (limited buffer size).
- To prevent this, most implementations follow the 3-duplicate-ACKs rule and retransmit the missing segment immediately.

First Scenario Normal operation



Second scenario

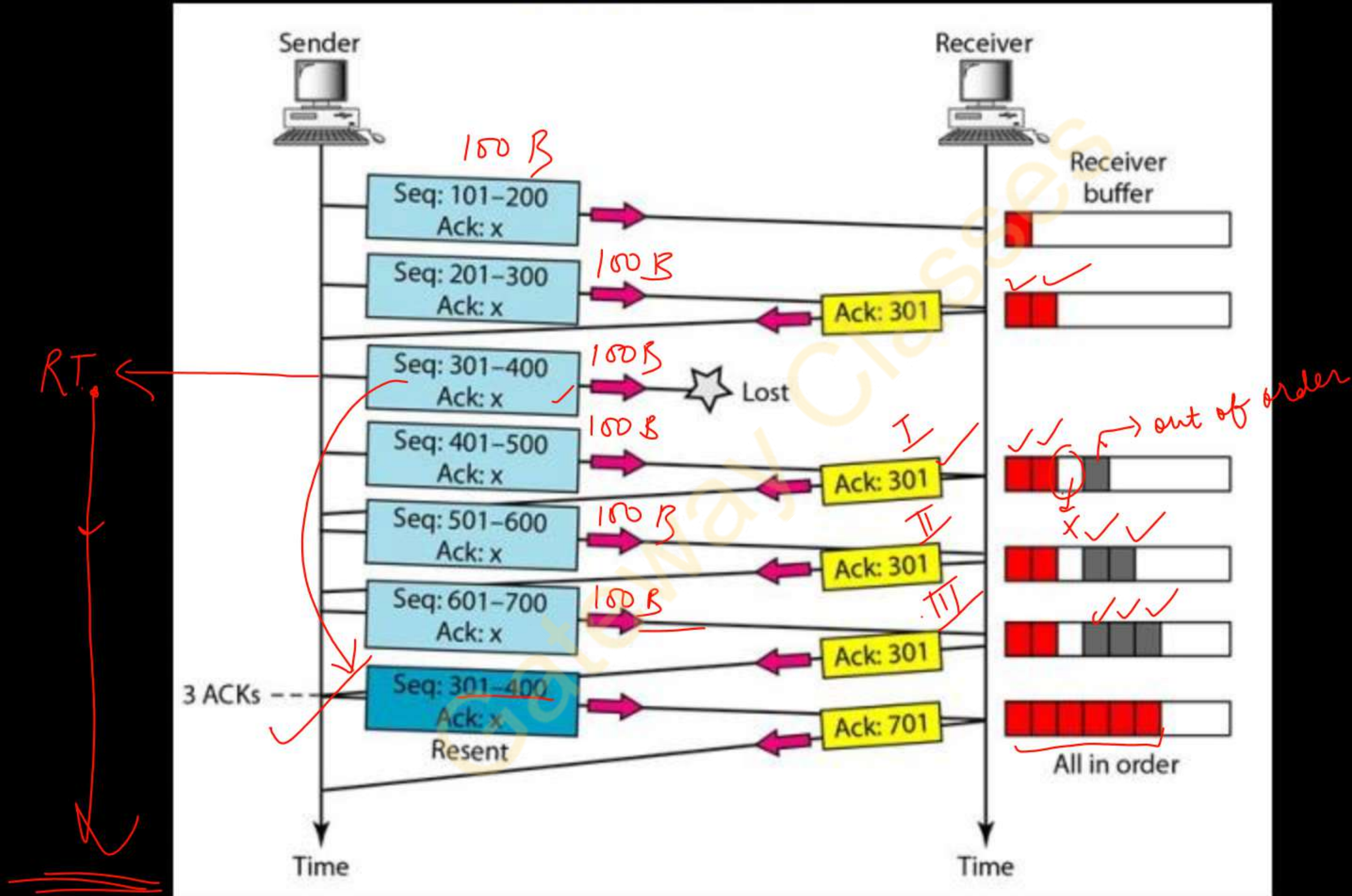
Lost segment



- Sender sends segments 1 and 2, which are acknowledged immediately by an ACK.
- Segment 3 is lost. The receiver receives segment 4, which is out of order.
- The receiver stores the data in the segment in its buffer but leaves a gap to indicate that there is no continuity in the data.
- The receiver immediately sends an acknowledgment to the sender, displaying the next byte it expects.
- Receiver stores bytes 801 to 900, but never delivers these bytes to the application until the gap is filled.
- The receiver TCP delivers only ordered data to the process.

Third Scenario

Fast retransmission



- Now RTO has a higher value
- When the receiver receives the fourth, fifth, and sixth segments, it sends an ack.
- The sender receives four acknowledgments with the same value (threeduplicates).
- Although the timer for segment 3 has not matured yet, the fast transmission requires that segment 3, the segment that is expected by all these acknowledgments, be resent immediately.
- Note that only one segment is retransmitted although four segments are not
- acknowledged.
- When the sender receives the retransmitted ACK, it knows that the four segments are safe and sound because acknowledgment is cumulative.

Congestion Control in TCP

Congestion Window

- Sender window size is determined by the available buffer space in the receiver (*rwnd*).
- Network (congestion in the network) also determines the size of the sender's window.
- Sender has two pieces of information: the receiver-advertised window size and congestion window size.
- The actual size of the window is the minimum of these two.

Actual window size = $\text{minimum}(rwnd, cwnd)$

Congestion Policy

Three phases: ① slow start, ② congestion avoidance, and ③ congestion detection.

- In the slow-start phase, the sender starts with a very slow rate of transmission, but increases the rate rapidly to reach a threshold. When the threshold is reached, the data rate is reduced to avoid congestion. Finally, if congestion is detected, the sender goes back to the slow-start or congestion avoidance phase based on how the congestion is detected.

Slow Start:

- Size of the congestion window (*cwnd*) starts with one maximum segment size (MSS).
- MSS is determined during connection establishment by using an option of the same name.
- The size of the window increases one MSS each time an ack is received.

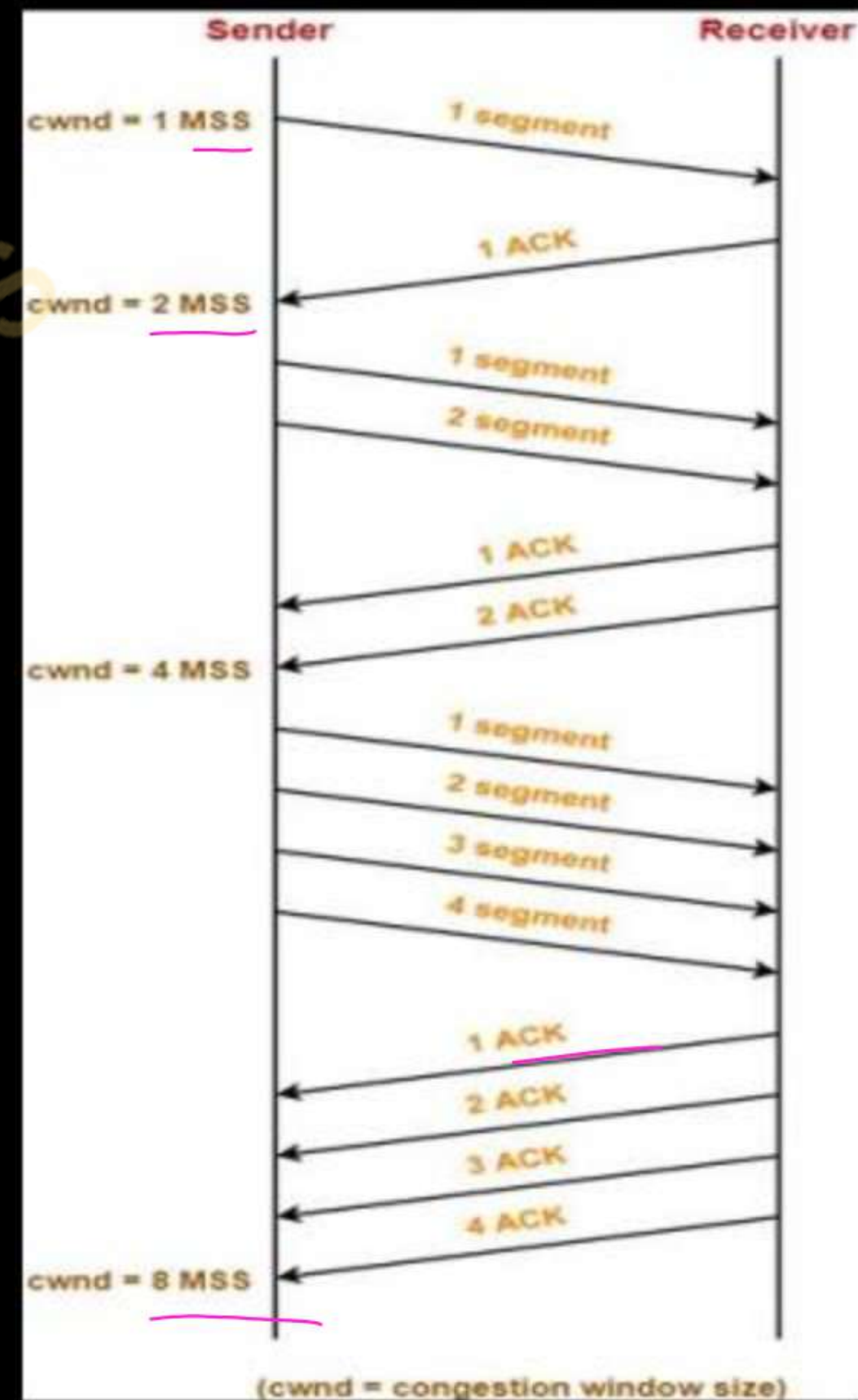
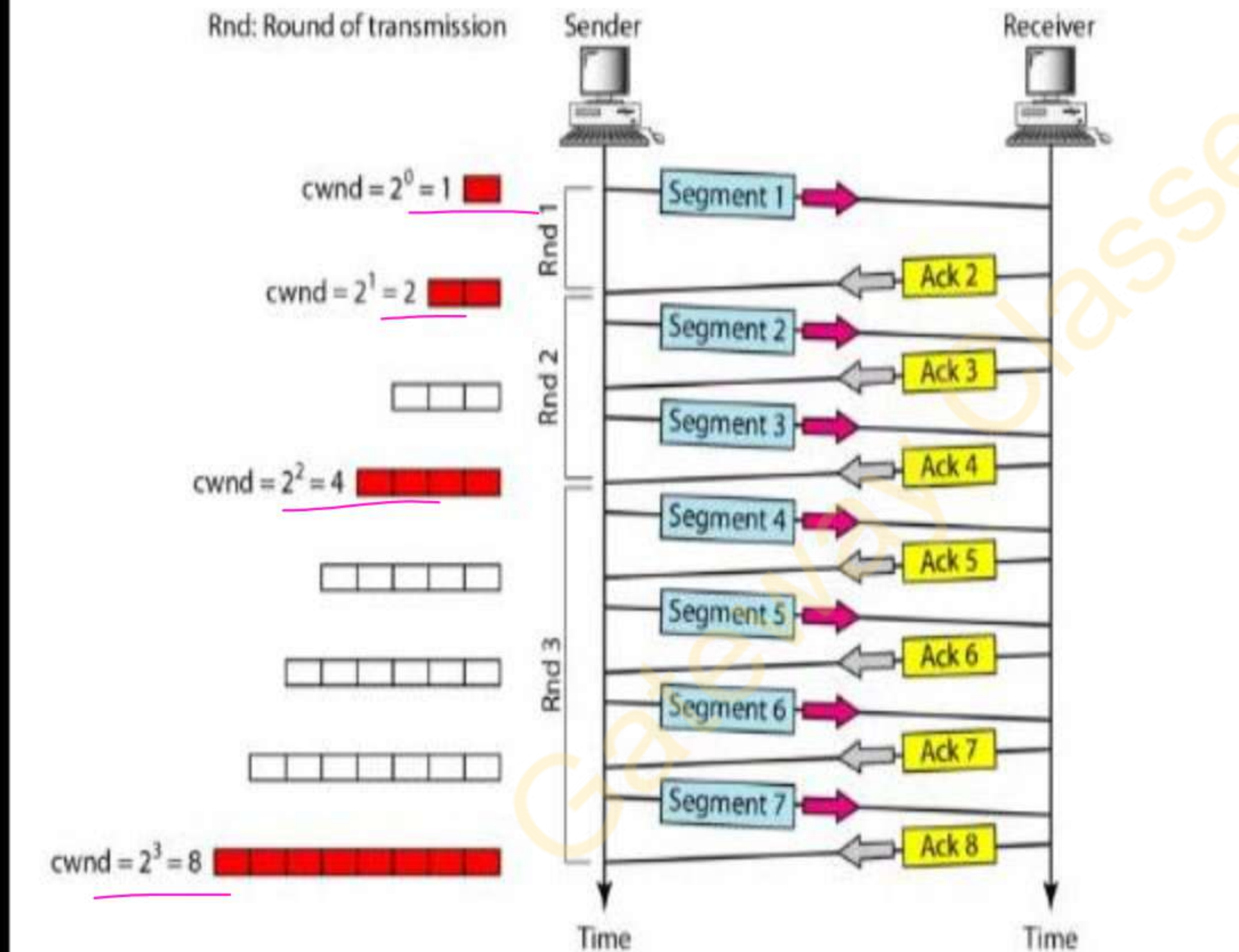
window starts slowly, but grows exponentially.

- The sender starts with $cwnd = 1$ MSS (sender can send only one segment). After receipt of ack for segment 1, size of the *cwnd* is increased by 1 (*cwnd* is now 2). Now two more segments can be sent and so on....

Start	$cwnd = 1$
After round 1	$cwnd = 2^1 = 2$
After round 2	$cwnd = 2^2 = 4$
After round 3	$cwnd = 2^3 = 8$

- In slow-start algo, size of congestion window increases exponentially until it reaches a threshold.
- In most implementations the value of *ssthresh* is 65,535 bytes.

Slow start, exponential increase



Congestion Avoidance: Additive Increase

- To avoid congestion before it happens; one must slow down this exponential growth.
- When size of congestion window reaches slow-start threshold, slow-start phase stops and additive phase begins.
- Now, each time the whole window of segments is acknowledged (one round), the size of congestion window is increased by 1.

Start

$$\text{cwnd}=\underline{1}$$

After round 1

$$\text{cwnd}=1+\underline{1}=2$$

After round 2

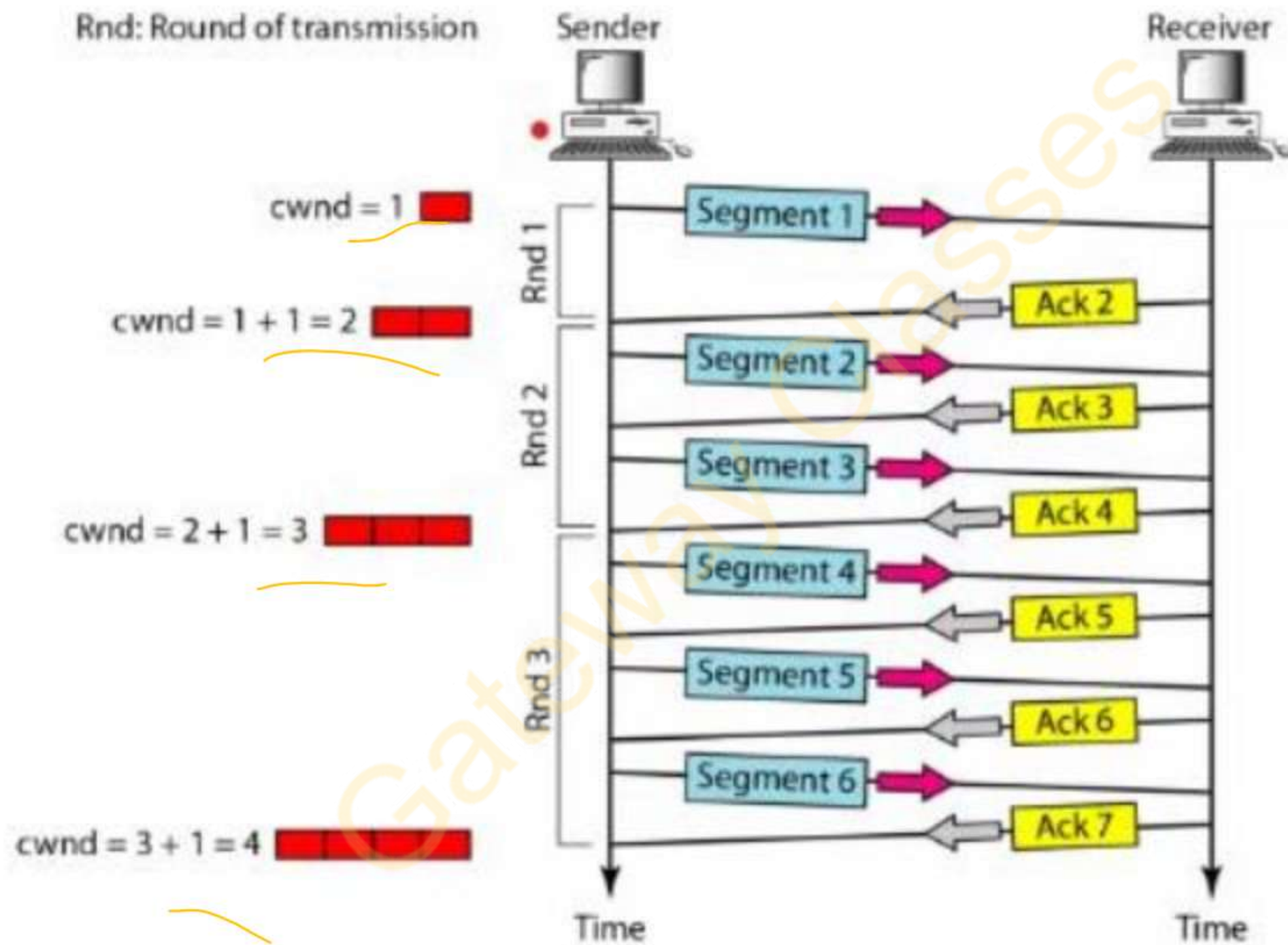
$$\text{cwnd}=2+\underline{1}=3$$

After round 3

$$\text{cwnd}=3+\underline{1}=4$$

Size of the congestion window increases additively until congestion is detected.

Congestion avoidance, additive increase



Congestion Detection: Multiplicative Decrease:

If congestion occurs, the congestion window size must be decreased.

➤ The only way the sender can guess that congestion has occurred is by the need to retransmit a segment.

➤ **Retransmission can occur when a timer times out or when three ACKs are received.**

1. If a time-out occurs, there is a stronger possibility of congestion; a segment has probably been dropped in the network.

- a. TCP sets value of threshold to one-half of the current window size.
- b. Sets *cwnd* to the size of one segment.
- c. Starts the slow-start phase again.

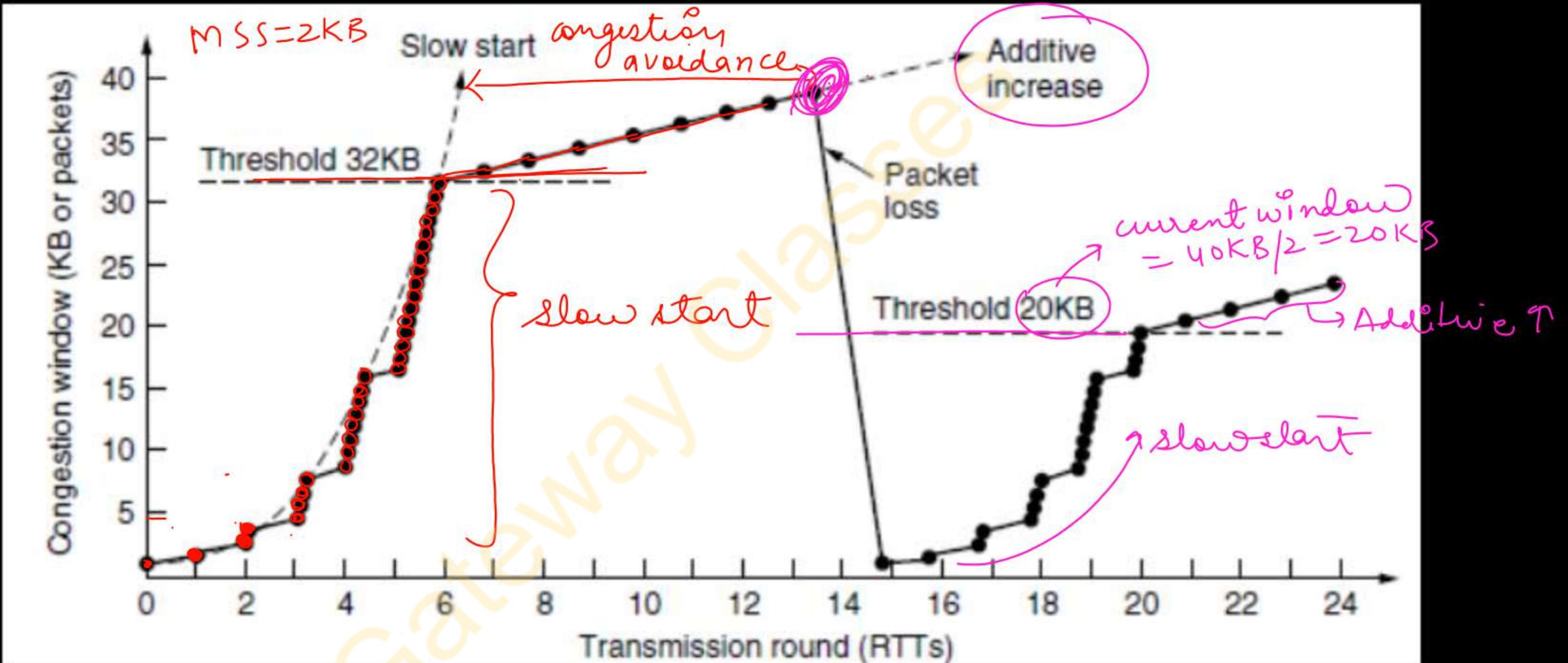
2. If three ACKs are received, there is a weaker possibility of congestion; a segment may have been dropped, but some segments after that may have arrived safely since three ACKs are received. This is called **fast transmission and fast recovery**.

- a. TCP sets the value of threshold to one-half of the current window size.
- b. Sets *cwnd* to value of the threshold (some implementations add three segment sizes to threshold).
- c. Starts the congestion avoidance phase.

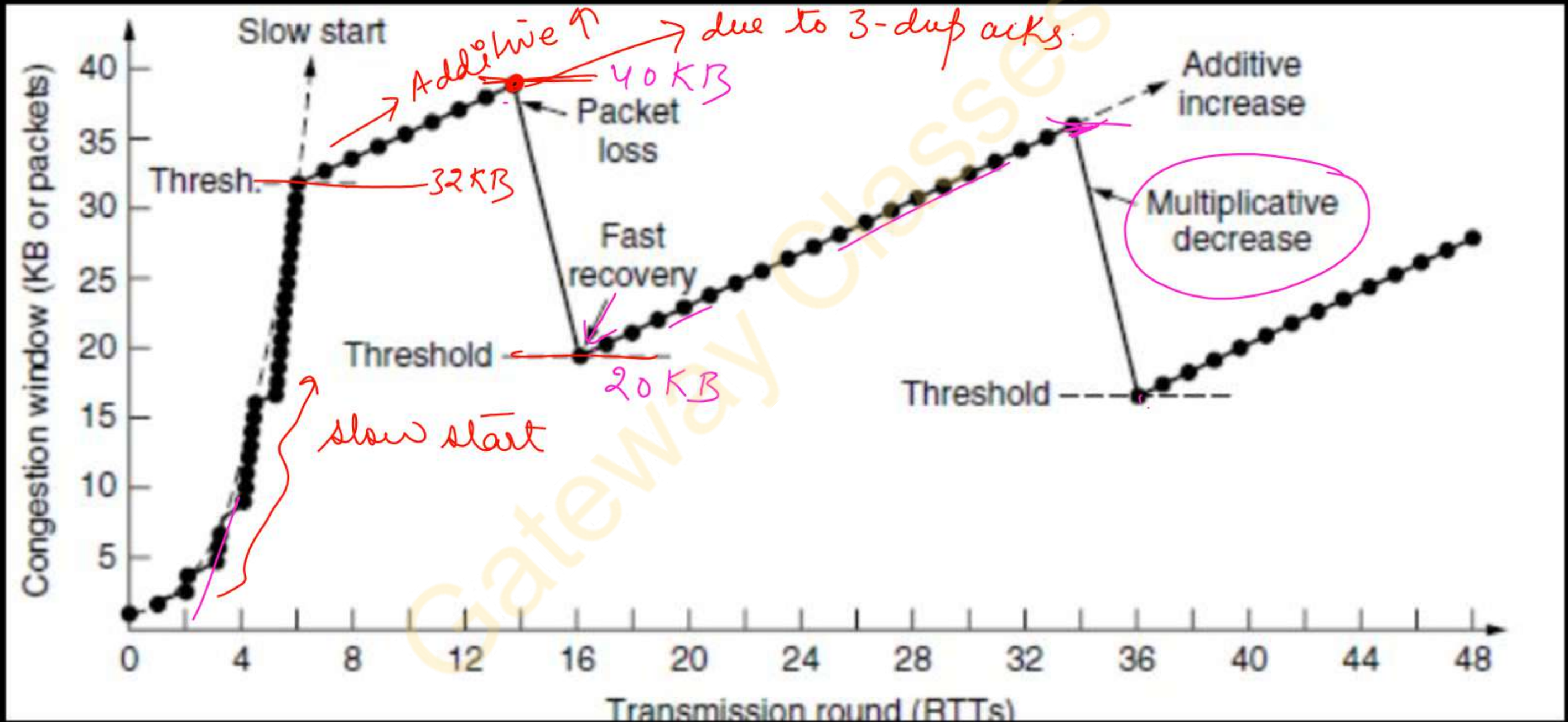
An implementations reacts to congestion detection in one of the following ways:

- If detection is by time-out, a new *slow-start* phase starts.
- If detection is by three ^{duplicate} ACKs, a new *congestion avoidance* phase starts.

Congestion Due to RTO



Congestion Due to 3-duplicate Acks



Q: Enumerate how the transport layer ensure that the complete message arrives at the destination and in the proper order.(AKTU 2016-17, 2017-18)

The transport layer in the OSI and TCP/IP models ensures that a complete message arrives at the destination accurately and in the proper order using the following mechanisms:

1. Segmentation and Reassembly
2. Sequencing
3. Acknowledgments (ACKs)
4. Error Detection and Correction
5. Retransmission (Automatic Repeat request - ARQ)
6. Flow Control (e.g., Sliding Window Protocol)
7. Connection Establishment and Termination (for TCP)

These functions are primarily handled by TCP (Transmission Control Protocol).

Q: How does transport layer perform duplication control? (AKTU 2017-18)

Duplication Control

Duplication Control is the fourth aspect of reliability. The transport layer guarantees that no duplicate data arrive at the destination. Sequence numbers are used to identify the lost packets; similarly, it allows the receiver to identify and discard duplicate segments.

AKTU PYQs

1. Explain the three-way handshaking protocol to establish the transport level connection (AKTU 2016-17, 2017-18)
2. Enumerate how the transport layer Ensure that the complete message arrives at the destination and in the proper order. (AKTU 2016-17, 2017-18)



AKTU

B.Tech 6th Semester
CS IT & Allied



Computer Networks

Unit-4: Lecture 4

Today's Target

- User Datagram Protocol(UDP) ✓
- Comparison between TCP and UDP ✓
- Quality of service ✓
 - Techniques to improve QoS
 - Scheduling
 - Traffic Shaping
 - Leaky Bucket
 - Token Bucket
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified



User Datagram Protocol (UDP)

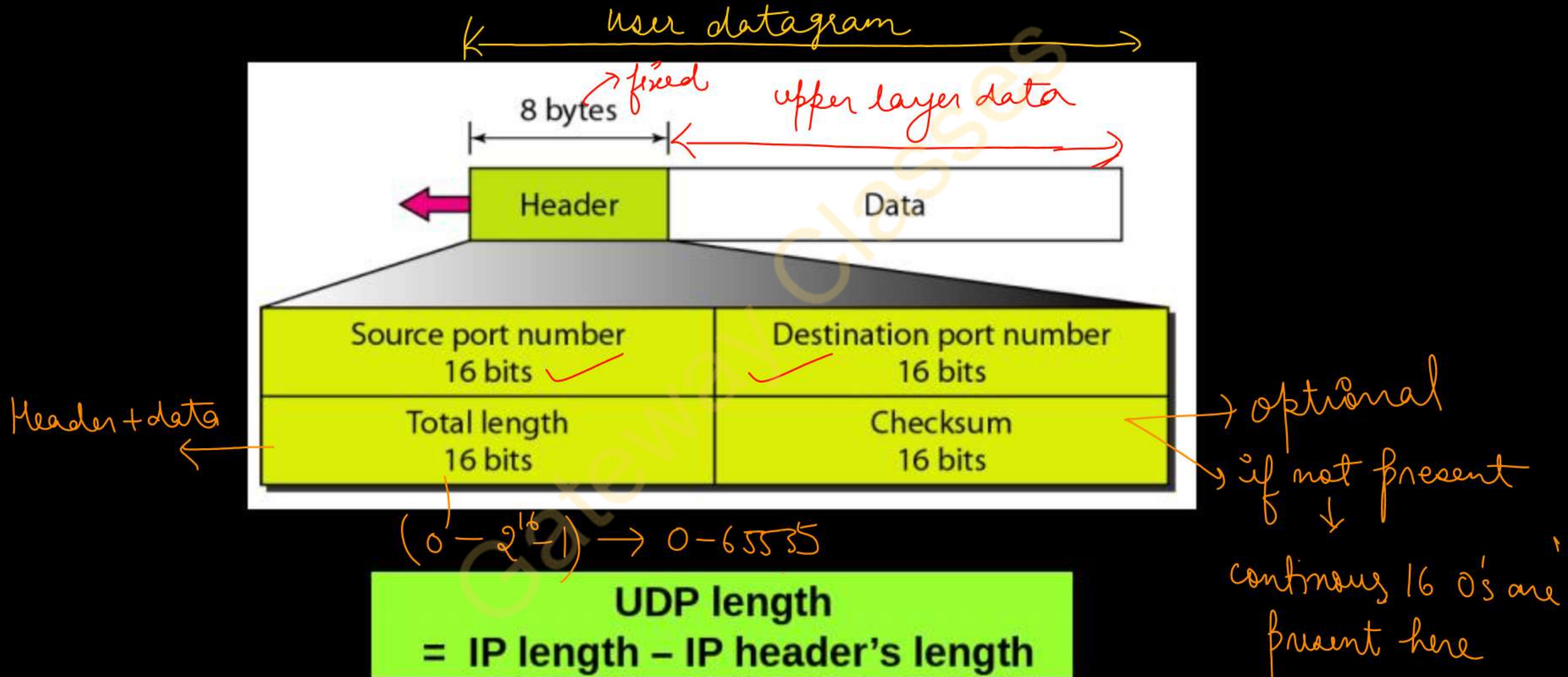
- UDP is a connectionless, unreliable transport protocol.
- It does not add anything to the services of IP except to provide process-to process communication.
- Performs very limited error checking.

Why UDP?

- UDP is a simple protocol using a minimum of overhead.
- If a process wants to send a small message and does not care much about reliability, it can use UDP.
- Sending a small message by using UDP takes much less interaction between the sender and receiver than using TCP.

User Datagram

UDP packets, called user datagrams, have a fixed-size header of 8 bytes.





IPv4

- 1) Header length
- 2) Total length

Total length - IP header

Source port number: 16 bits long port number used by the process running on the source host.

Destination port number: 16 bits long port number used by the process running on the destination host.

Length: This is a 16-bit field that defines the total length of the user datagram, header plus data.

The 16 bits can define a total length of 0 to 65,535 bytes.

Length field in a UDP user datagram is not necessary as user datagram is encapsulated in IP datagram.

A field in IP datagram defines total length and another field defines header length.

$$\text{UDP length} = \text{IP length} - \text{IP header's length}$$

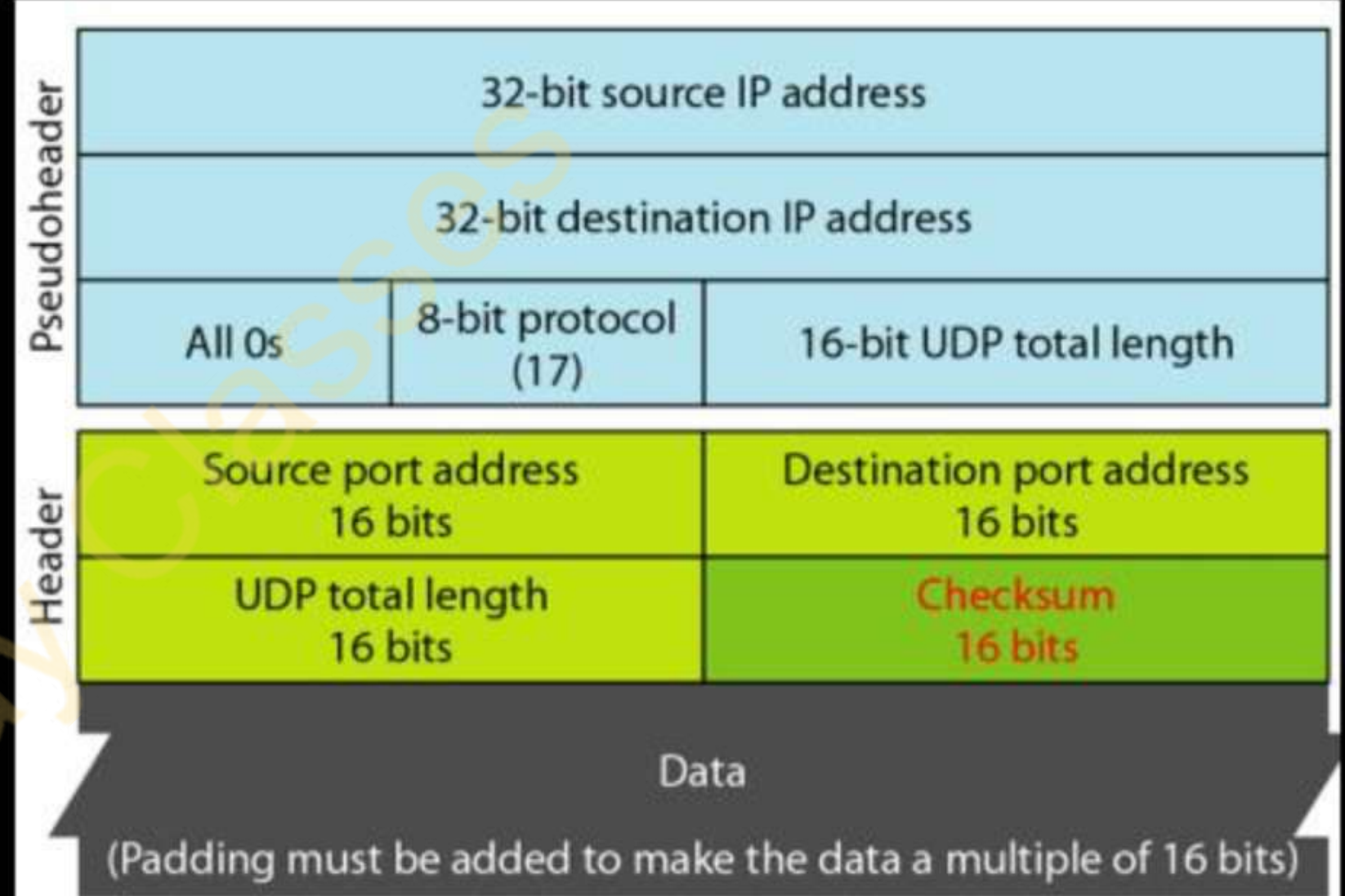
Checksum:

Here checksum includes three sections:

1. a pseudo header
2. UDP header
3. data coming from application layer

Pseudo header is part of IP header of packet in which user datagram is to be encapsulated with some fields filled with 0s.

Pseudo header for checksum calculation



- If the checksum does not include the pseudo header, a user datagram may arrive safe and sound.
- If IP header is corrupted, it may be delivered to the wrong host.
- The protocol field is added to ensure that the packet belongs to UDP.
- The value of the protocol field for UDP is 17.
- If this value is changed during transmission, the checksum calculation at the receiver will detect it and UDP drops the packet. It is not delivered to the wrong protocol.

UDP Operation

Connectionless Services:

- Each user datagram sent by UDP is an independent datagram.
- The user datagrams are not numbered.
- No connection establishment and no connection termination.
- Process that uses UDP cannot send a stream of data to UDP and expect UDP to chop them into different related user datagrams. Only those processes sending short messages should use UDP.

Flow and Error Control:

- UDP is a simple, unreliable transport protocol.
- No flow control and no window mechanism.
- No error control mechanism in UDP except for the checksum.
- When receiver detects an error through checksum, user datagram is silently discarded.

Encapsulation and Decapsulation:

To send a message from one process to another, the UDP protocol encapsulates and decapsulates messages in an IP datagram.

Queuing:

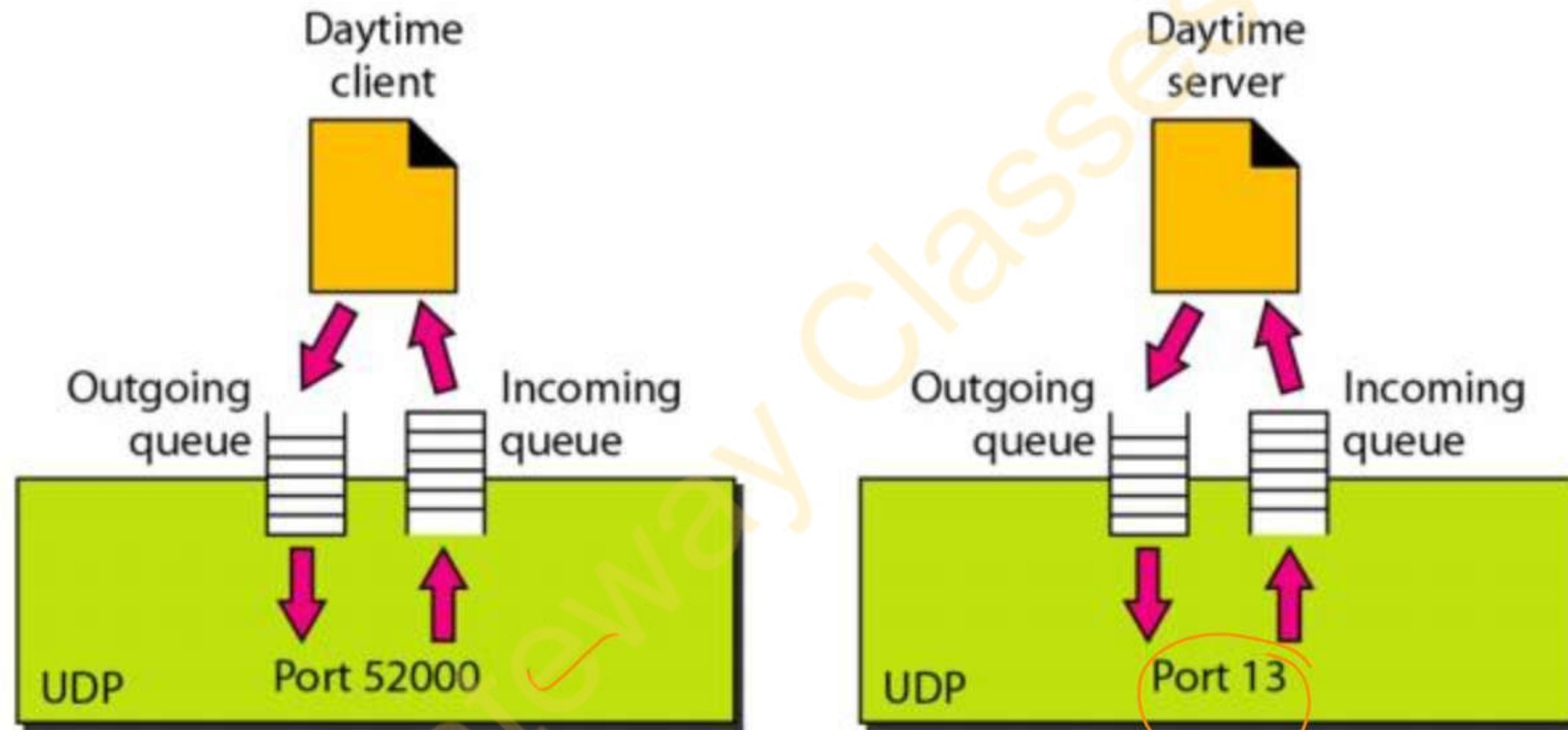
At client site, when a process starts, it obtains one port number, one outgoing and one incoming queue.

- Client process send messages to outgoing queue, UDP removes them one by one, adds UDP header, delivers them to IP.
- If outgoing queue overflows, OS ask client process to wait before sending any more messages.
- When a message arrives for a client, UDP sends the received datagram to end of incoming queue.
- If there is no such queue or incoming queue overflows, UDP discards user datagram and asks ICMP protocol to send a *port unreachable* message to the server.

- **At the server site**, a server asks for incoming and outgoing queues, using its well-known port, when it starts running. The queues remain open as long as the server is running.
- When a message arrives for server, UDP sends received user datagram to end of incoming queue.
- If there is no such queue, or incoming queue overflow, UDP discards the user datagram and asks the ICMP protocol to send a port unreachable message to the client.
- When a server wants to respond to a client, it sends messages to the outgoing queue, using the source port number specified in the request.
- UDP removes the messages one by one and, after adding the UDP header, delivers them to IP.
- If outgoing queue overflows, OS asks the server to wait before sending any more messages.



Operation of UDP



Applications of UDP (Imp)

1. Suitable for a process for simple request-response communication without flow and error control.
2. Not usually used for a process that needs to send bulk data.
3. UDP is suitable for a process with internal flow and error control mechanisms.
4. UDP is a suitable for multicasting(embedded in the UDP software).
5. Normally used for real-time applications which can not tolerate uneven delays.
6. UDP is used for management processes. (usually sends command/Response)
7. UDP is used for some route updating protocols such as Routing Information Protocol (RIP)
8. Used in VoIP (Voice over Internet Protocol) services, such as WhatsApp for real-time voice communication.
9. Used in DNS_for query/response messages as small DNS queries require a quick response time.
10. Used in DHCP to dynamically assign IP addresses to devices on a network.

Difference between TCP and UDP (Imp)

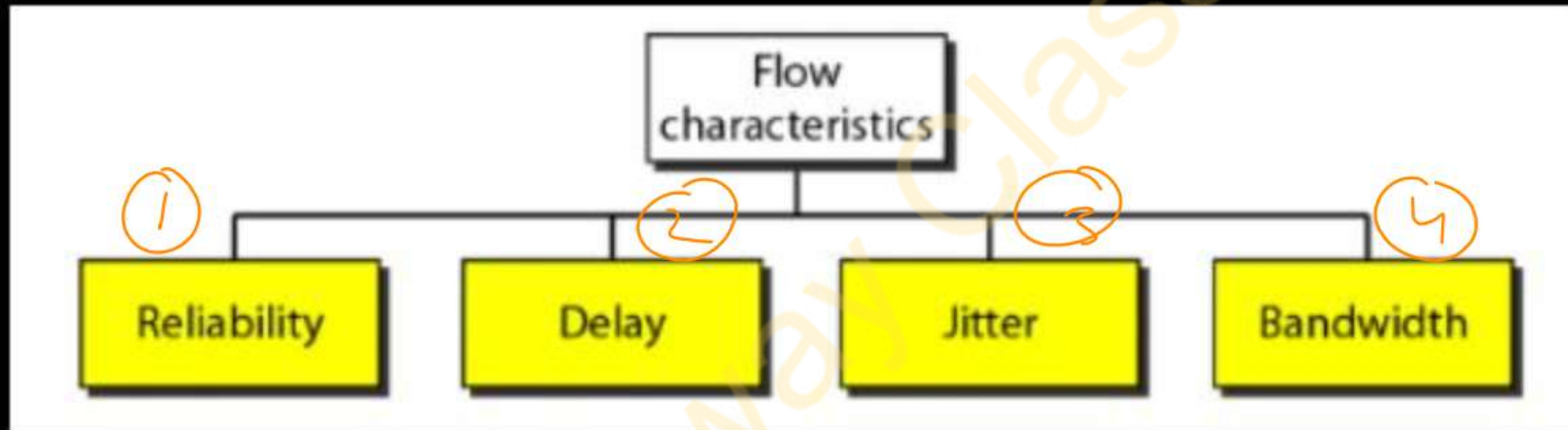
Basis	TCP	UDP
Type of Service	connection-oriented	Connection-less
Reliability	Reliable (guaranteed delivery of data to the destination)	Unreliable(delivery of data to destination not guaranteed) .
Error checking	extensive error-checking. provides flow control and acknowledgment of data.	basic error-checking mechanism using <u>checksums</u> .
Acknowledgment	An ack segment is present.	No acknowledgment segment.
Sequence	Sequencing of data, packets arrive in order at the receiver.	No sequencing of data in UDP. If the order is required, it has to be managed by the application
Speed	Slower than UDP.	UDP is faster
Retransmission	Retransmission of lost packets	No retransmission of lost packets
Header Length	(20-60) bytes variable length header.	8 bytes fixed-length header.

Basis	TCP	UDP
Handshaking Techniques	Uses handshakes such as SYN, ACK, SYN-ACK	It's a connectionless protocol i.e. No handshake
Broadcasting	Doesn't support Broadcasting.	Supports Broadcasting.
Protocols	Used by <u>HTTP</u> , <u>HTTPs</u> , <u>FTP</u> , <u>SMTP</u> and <u>Telnet</u> .	UDP is used by <u>DNS</u> , <u>DHCP</u> , <u>TFTP</u> , <u>SNMP</u> , <u>RIP</u> , and <u>VoIP</u> .
Stream Type	The TCP connection is a byte stream.	UDP connection is a message stream.
Overhead	Low but higher than UDP.	Very low.

Quality of Service

Quality of service (QoS) is an internetworking parameter a flow seeks to attain.

Flow Characteristics: Four characteristics are attributed to a flow: reliability, delay, jitter, and bandwidth



1. Reliability:

- Lack of reliability means losing a packet or ack, which entails retransmission.
- For example, it is more important that electronic mail, file transfer, and Internet access have reliable transmissions than telephony or audio conferencing.

2. Delay:

- Applications can tolerate delay in different degrees.
- Here, telephony, audio conferencing, video conferencing, and remote log-in need minimum delay, while delay in file transfer or e-mail is less important.

3. Jitter:

- Variation in delay for packets belonging to the same flow.
- if four packets depart at times 0, 1, 2, 3 and arrive at 20, 21, 22, 23, all have same delay of 20 units of time. But if these packets arrive at 21, 23, 21, 28, they have different delays: 21, 22, 19, and 24.

- For applications such as audio and video, the jitter is not acceptable;
- For these applications, it does not matter if the packets arrive with a short or long delay as long as the delay is the same for all packets.
 - High jitter means the difference between delays is large;
 - low jitter means the variation is small.

4. Bandwidth:

- Different applications need different bandwidths.
- In video conferencing we need to send millions of bits per second to refresh a color screen.
- Total number of bits in an e-mail may not reach even a million.

Techniques to improve QoS

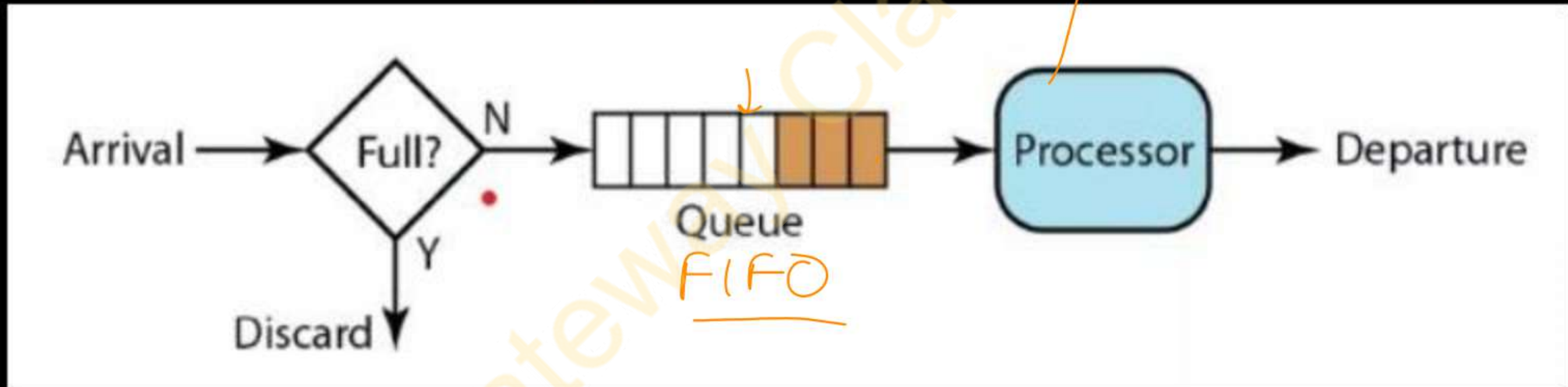
- ✓ 1. Scheduling
 - FIFO
 - Priority
 - weighted fair queue.
- ✓ 2. Traffic Shaping
 - (Congestion Control)
 - a) Leaky Bucket ✓
 - b) Token Bucket ✓
- ✓ 3. Resource Reservation
- ✓ 4. Admission Control

Scheduling: Packets from different flows arrive at a switch or router for processing. A good scheduling technique treats different flows in a fair and appropriate manner. Scheduling techniques :

- ✓ a) FIFO queuing
- ✓ b) priority queuing
- ✓ c) weighted fair queuing

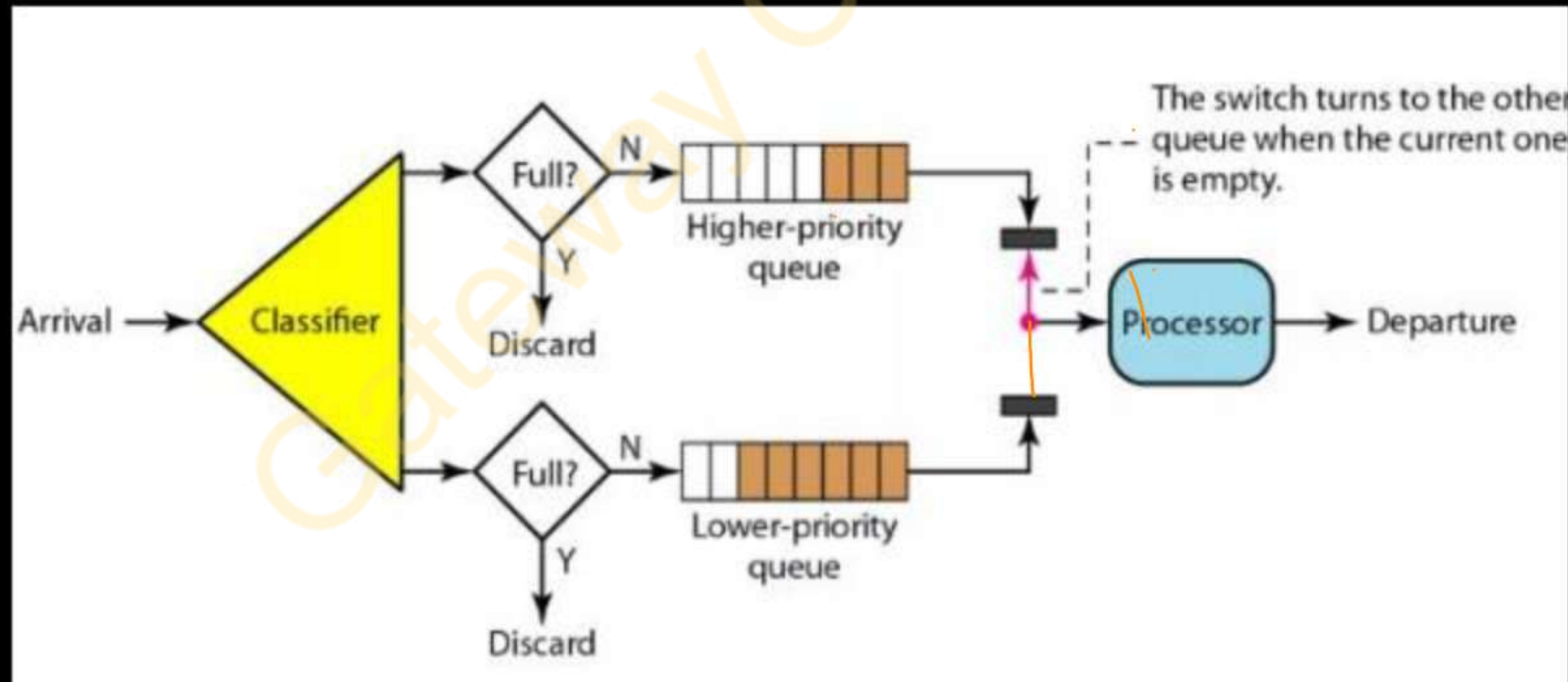
FIFO Queuing:

- Packets wait in a buffer (queue) until the node (router or switch) is ready to process them.
- If the average arrival rate is higher than the average processing rate, the queue will fill up and new packets will be discarded.



Priority Queuing: Packets are first assigned to a priority class. Each priority class has its own queue.

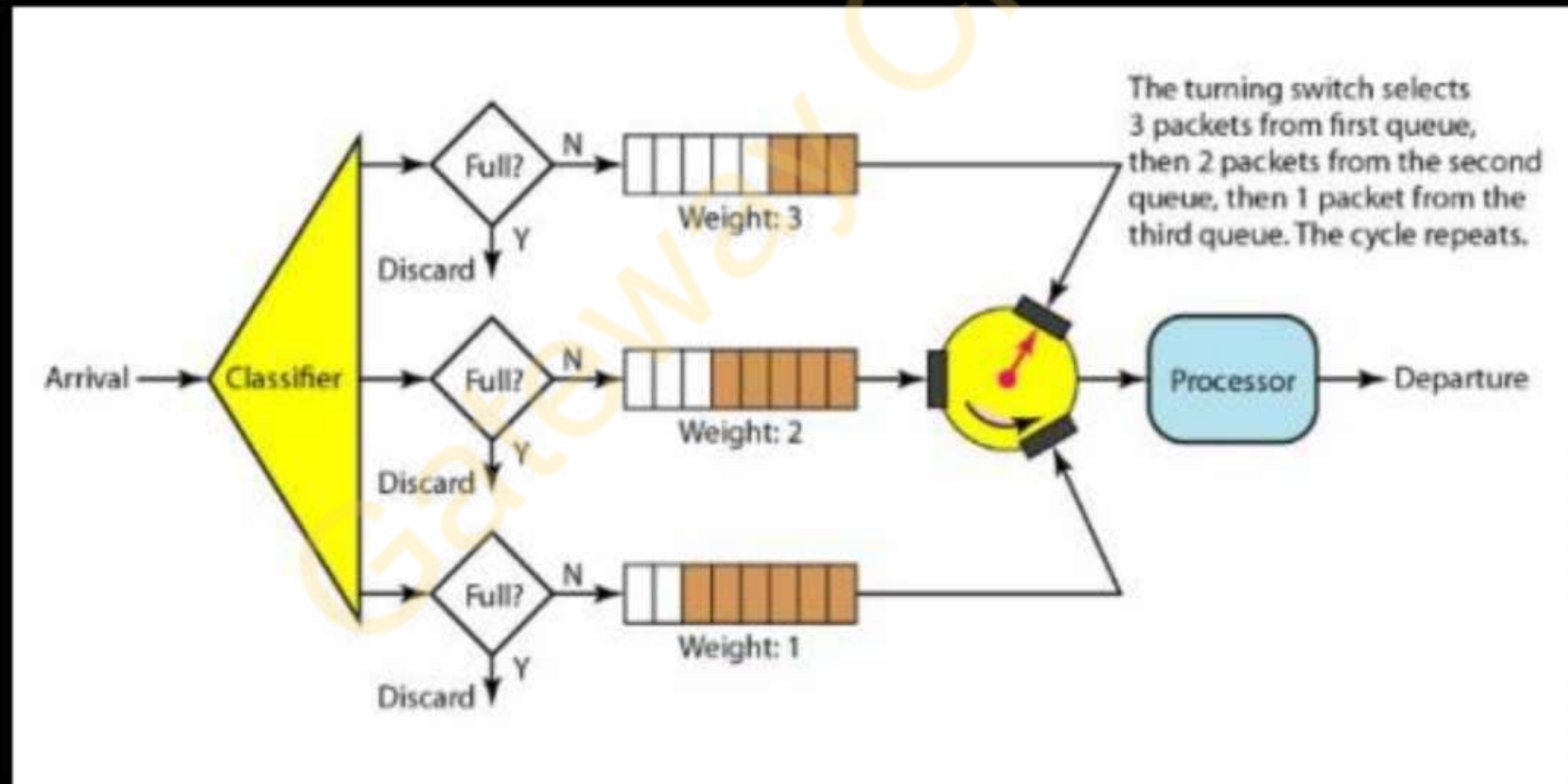
- Packets in highest-priority queue are processed first and ones in lowest-priority queue are processed last. System serves a queue until it is empty.
- Better QoS than FIFO queue as higher priority traffic(multimedia) reach destination with less delay.
- If there is a continuous flow in a high-priority queue, the packets in lower-priority queues will never have a chance to be processed. This is a condition called *starvation*.



Weighted Fair Queuing

- The queues are weighted based on the priority of the queues; higher priority means a higher weight.
- The system processes packets in each queue in a round-robin fashion with the number of packets selected from each queue based on the corresponding weight.

Example: if weights are 3, 2, and 1, 3 packets are processed from first queue, 2 from second queue, and 1 from third queue. If system doesn't impose priority, all weights are equal leading to fair queuing with priority.



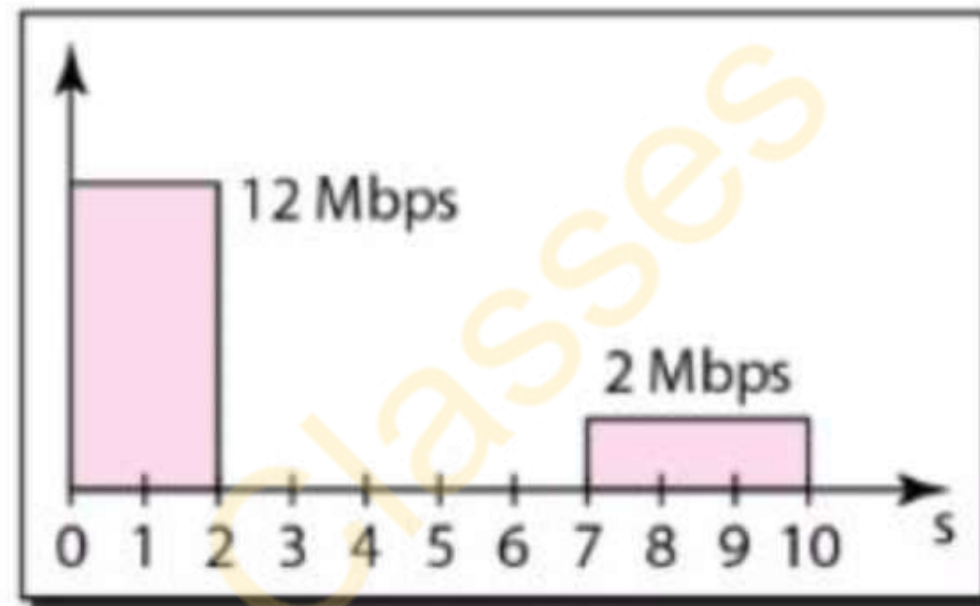
Traffic Shaping (Imp)

Traffic shaping is a mechanism to control the amount and the rate of the traffic sent to the network.

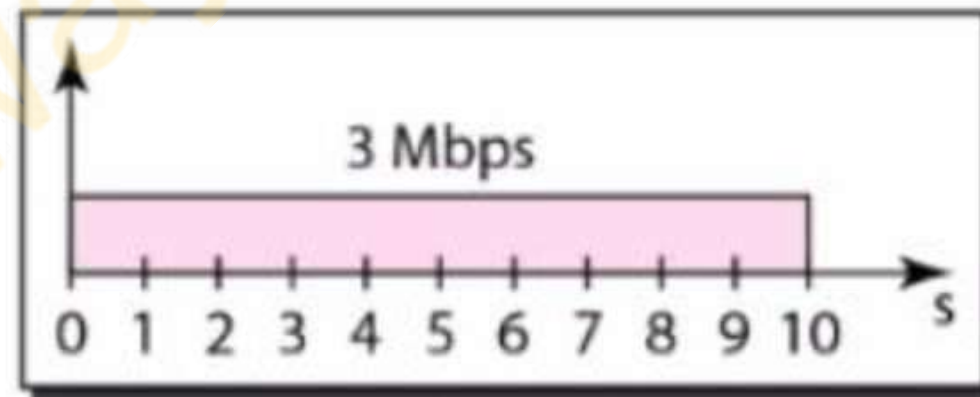
Two techniques can shape traffic: leaky bucket^① and token bucket^②.

Leaky Bucket:

- If a bucket has a small hole at the bottom, the water leaks from the bucket at a constant rate as long as there is water in the bucket.
- The rate at which the water leaks does not depend on the rate at which the water is input to the bucket unless the bucket is empty.
- The input rate can vary, but the output rate remains constant.
- Similarly, in networking, a technique called leaky bucket can smooth out bursty traffic.
- Bursty chunks are stored in the bucket and sent out at an average rate.

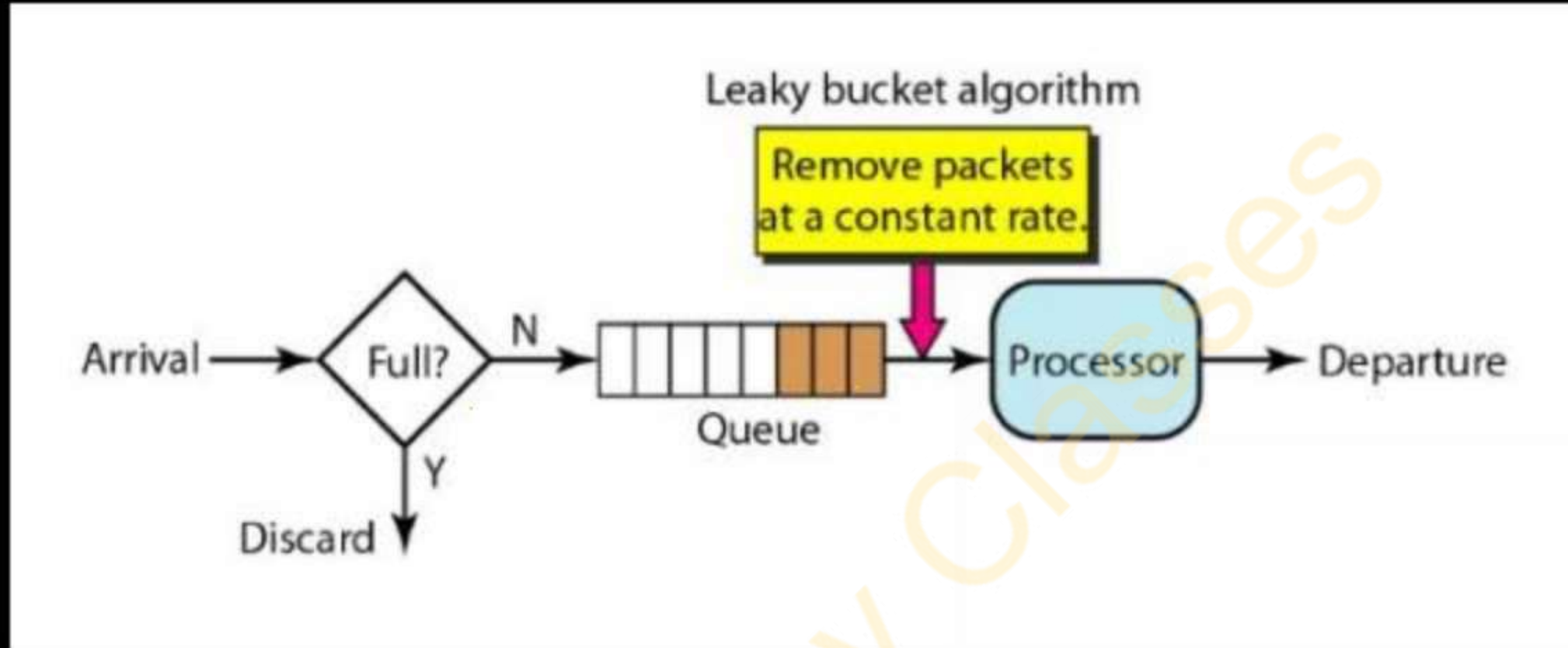


Bursty data



Fixed-rate data

Leaky bucket implementation



- FIFO queue holds the packets.
- If the traffic consists of fixed-size packets, the process removes a fixed number of packets from the queue at each tick of the clock.
- If traffic consists of variable-length packets, fixed output rate must be based on no. of bytes or bits.

The following is an algorithm for variable-length packets:

1. Initialize a counter to n at the tick of the clock.
2. If n is greater than the size of the packet, send the packet and decrement the counter by the packet size. Repeat this step until n is smaller than the packet size.
3. Reset the counter and go to step 1.

- A leaky bucket algorithm shapes bursty traffic into fixed-rate traffic by averaging the data rate.
- It may drop the packets if the bucket is full.

Drawback:

- The leaky bucket algorithm does not credit an idle host. ✓
- For example, if a host is not sending for a while; its bucket becomes empty. Now if the host has bursty data, the leaky bucket allows only an average rate. The time when the host was idle is not taken into account.

Token Bucket:

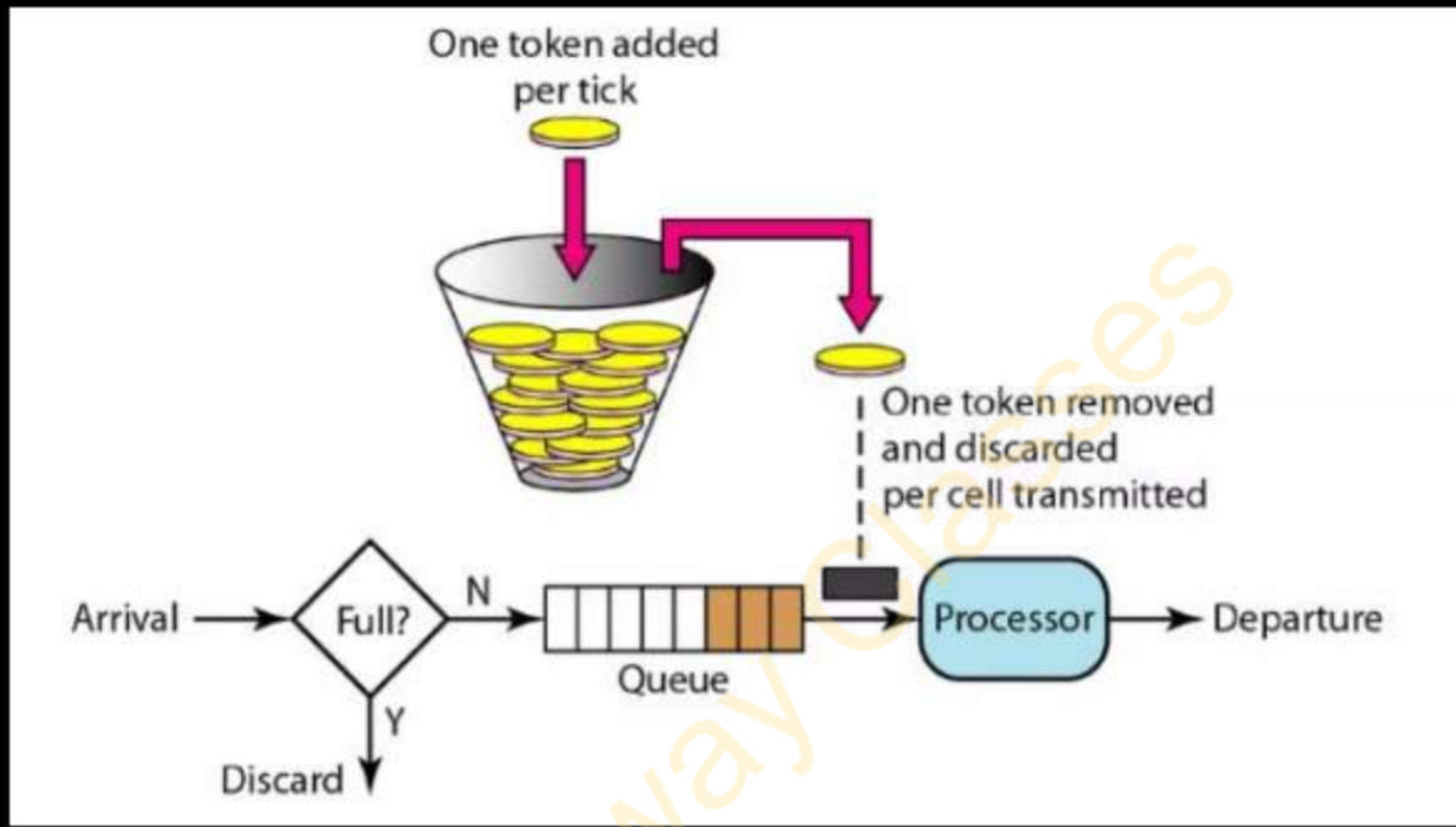
- Token bucket algorithm allows idle hosts to accumulate credit for the future in the form of tokens.
- For each tick of the clock, the system sends n tokens to the bucket.
- The system removes one token for every cell (or byte) of data sent.

1 Token / byte

Example: if n is 100 and the host is idle for 100 ticks, the bucket collects 10,000 tokens.

- Now the host can consume all these tokens in one tick with 10,000 cells, or the host takes 1000 ticks with 10 cells per tick. **The host can send bursty data as long as the bucket is not empty**

- Token bucket can be implemented with a counter. Token is initialized to zero.
- Each time a token is added, the counter is incremented by 1.
- Each time a unit of data is sent, the counter is decremented by 1.
 - When the counter is zero, the host cannot send data.
- **The token bucket allows bursty traffic at a regulated maximum rate.**



Combining Token Bucket and Leaky Bucket:

- The two techniques can be combined to credit an idle host and regulate the traffic.
- The leaky bucket is applied after the token bucket; the rate of the leaky bucket needs to be higher than the rate of tokens dropped in the bucket.

Differences Between Token Bucket and Leaky Bucket Algorithm (Imp)

Feature	Token Bucket Algorithm	Leaky Bucket Algorithm
Traffic Handling	Allows for burst traffic by saving tokens	Smooths traffic by enforcing a constant rate
Concept	Uses tokens to control data transmission	Uses a leaky bucket to control data transmission
Data Dropping	Excess tokens are discarded if the bucket overflows	Excess data is discarded if the bucket overflows
Rate Flexibility	More flexible with bursts	More rigid, enforces a steady rate
Use Case	Suitable for bursty traffic and rate limiting	Suitable for steady rate enforcement and shaping
Implementation Complexity	More complex	Simpler
Buffering	Data can be buffered until tokens are available	Limited buffering, excess data is dropped

Resource Reservation:

A flow of data needs resources such as a buffer, bandwidth, CPU time, and so on.

The QoS is improved if these resources are reserved beforehand.

Admission Control:

- Admission control refers to the mechanism used by a router, or a switch, to accept or reject a flow based on predefined parameters called flow specifications.
- Before a router accepts a flow for processing, it checks the flow specifications to see if its capacity (in terms of bandwidth, buffer size, CPU speed, etc.) and its previous commitments to other flows can handle the new flow.

AKTU PYQs

1. Differentiate TCP and UDP in context of the header format.(AKTU 2022-23)
2. Write detailed note on “TCP vs UDP”. (AKTU 2021-22)
3. Define QoS.(AKTU 2022-23)
4. Define Token Bucket (AKTU 2023-24)

Thank
you